Année universitaire 2026 – 2027

Module : Langage de modélisation

UML

Unified Modeling Language

Application de gestion d'un complexe sportif

Thématique : Sport & Event Management

Rapport de projet

Réalisé par

Haroun Al Rachid Ben Nasr

Youssef Hammi

Mohamed Yassine Naccache

Nour Ben Amor

Mohamed Amine Khadhraoui

Encadrante

Mme Mahfoudhi Ryma

Sommaire

Partie I : UML 1

1 La modélisation UML 2

2 L'intelligence artificielle et UML 2

2.1 Assistance à la réflexion et génération d'idées 2

2.2 Importance d'un prompt bien cadré 3

2.3 Limites de l'IA dans la modélisation UML 3

2.4 Types d'erreurs possibles 3

2.5 Importance de la validation humaine 4

3 Comparaison des outils UML 5

Partie II : Projet 6

1 Introduction 7

1.1 Contexte du projet 7

1.2 Problématique 7

1.3 Motivation du projet 7

1.4 Objectifs du projet 7

1.5 Périmètre du projet 7

2 Présentation générale de la solution 8

2.1 Description de l'application 8

2.2 Principe de fonctionnement 8

2.3 Valeur ajoutée de la solution 8

2.4 Utilisateurs et acteurs du système 9

3 Étude des besoins 9

4 Diagramme de cas d'utilisation global 10

5 Diagramme de cas d'utilisation : Authentification 11

6 Diagrammes de cas d'utilisation détaillés : Client 12

6.1 Gérer ses réservations . . . 13

6.2 S'inscrire à un cours ou tournoi . . . 14

6.3 Gérer son abonnement . . . 15

7 Diagrammes de cas d'utilisation détaillés : Coach 15

7.1 Gérer ses cours . . . 16

7.2 Gérer ses participants . . . 16

8 Diagrammes de cas d'utilisation détaillés : Admin 17

8.1 Gérer les installations . . . 17

8.2 Gérer les tarifs . . . 18

8.3 Gérer les utilisateurs . . . 18

8.4 Gérer les tournois . . . 19

9 Diagrammes de cas d'utilisation détaillés : Réceptionniste 19

9.1 Gérer les réservations sur place . . . 20

9.2 Gérer les paiements sur place . . . 20

10 Diagramme de classes 21

10.1 Diagramme de classes global . . . 21

10.2 Détail des classes . . . 23

11 Diagrammes de séquence 33

11.1 Authentification à deux facteurs . . . 34

11.2 Créer un tournoi . . . 35

11.3 Gérer les paiements sur place . . . 37

12 Diagrammes d'activités 38

12.1 Coach : consulter son planning . . . 38

12.2 Coach : gérer ses cours . . . 39

12.3 Administrateur . . . 40

12.4 Réceptionniste : gérer les paiements sur place 41

Table des figures

1 Diagramme de cas d'utilisation global du système de gestion du complexe sportif . . . 10
2 Diagramme de cas d'utilisation : authentification à deux facteurs (2FA) . . 11
3 Diagramme de cas d'utilisation : gérer ses réservations . . . 13
4 Diagramme de cas d'utilisation : s'inscrire à un cours ou tournoi . . . 14
5 Diagramme de cas d'utilisation : gérer son abonnement . . . 15
6 Diagramme de cas d'utilisation : gérer ses cours . . . 16
7 Diagramme de cas d'utilisation : gérer ses participants . . . 16
8 Diagramme de cas d'utilisation : gérer les installations . . . 17
9 Diagramme de cas d'utilisation : gérer les tarifs . . . 18
10 Diagramme de cas d'utilisation : gérer les utilisateurs . . . 18
11 Diagramme de cas d'utilisation : gérer les tournois . . . 19
12 Diagramme de cas d'utilisation : gérer les réservations sur place . . . 20
13 Diagramme de cas d'utilisation : gérer les paiements sur place . . . 20
14 Diagramme de classes global du système de gestion du complexe sportif . . 22
15 Classe abstraite `Utilisateur` . . . 23
16 Classe `Client` . . . 25
17 Classe `Coach` . . . 25
18 Classe `Receptionniste` . . . 26
19 Classe abstraite `Activite` . . . 26
20 Classe `Cours` . . . 27
21 Classe `Tournoi` . . . 27
22 Classe `Inscription` . . . 28
23 Classe `Reservation` . . . 28
24 Classe `Installation` . . . 29
25 Classe `Horaire` . . . 29
26 Classe `Creneau` . . . 30
27 Classe `Abonnement` . . . 30
28 Classe `Tarif` . . . 31

29 Classe `Promotion` . . . 31
30 Classe `Paiement` . . . 32
31 Classe `Notification` . . . 32
32 Classe `CodeVerification` . . . 33
33 Diagramme de séquence : authentification à deux facteurs (2FA) . . . 34
34 Diagramme de séquence : créer un tournoi . . . 35
35 Diagramme de séquence : gérer les paiements sur place . . . 37
36 Diagramme d'activités : consulter son planning (Coach) . . . 38
37 Diagramme d'activités : gérer ses cours (Coach) . . . 39
38 Diagramme d'activités : Administrateur . . . 40
39 Diagramme d'activités : gérer les paiements sur place (Réceptionniste) . . . 41

Liste des tableaux

1 Les types de diagrammes UML . . . 2
2 Types d'erreurs possibles de l'IA en modélisation UML . . . 4
3 Comparaison des outils UML . . . 5
4 Périmètre du projet . . . 8
5 Utilisateurs du système et leurs responsabilités . . . 9
6 Acteurs et cas d'utilisation du système . . . 11
7 Éléments du diagramme d'authentification . . . 12
8 Organisation du diagramme de classes en paquetages . . . 21
9 Principales relations du diagramme de classes . . . 24
10 Récapitulatif des diagrammes de séquence . . . 33

Partie I

UML

Atelier de recherche

1 La modélisation UML

UML (*Unified Modeling Language*) est un langage de modélisation standard utilisé pour représenter visuellement la structure et le fonctionnement d'un système logiciel. Il permet de décrire les différents aspects d'une application à travers plusieurs types de diagrammes, comme les diagrammes de cas d'utilisation, de classes, de séquence ou d'activités.

La modélisation UML présente plusieurs intérêts dans un projet logiciel. Elle permet tout d'abord de mieux comprendre et analyser les besoins avant de commencer le développement. Les diagrammes offrent également un moyen de communication commun entre les différents membres d'une équipe, notamment les développeurs, les analystes et les clients.

UML permet aussi d'identifier plus facilement les erreurs ou les incohérences dans la conception avant l'implémentation du logiciel. Enfin, les modèles UML peuvent servir de documentation du projet et faciliter sa maintenance et son évolution.

Table 1. Les types de diagrammes UML

Vue	Rôle	Diagrammes
Fonctionnelle	Ce que fait le système et pour qui	Cas d'utilisation
Structurelle (statique)	Comment le système est organisé	Classes, objets, paquetages, composants, déploiement, structure composite
Comportementale (dynamique)	Comment le système fonctionne dans le temps	Séquence, communication, activité, états-transitions, timing, vue d'ensemble des interactions

2 L'intelligence artificielle et UML

L'intelligence artificielle peut être utilisée comme un outil d'assistance dans le travail de modélisation UML. Elle ne remplace pas l'analyste ou le développeur, mais peut faciliter certaines étapes de réflexion et de conception.

2.1 Assistance à la réflexion et génération d'idées

Une IA générative peut analyser une description textuelle d'un système et proposer des éléments de modélisation. Par exemple, à partir de la description d'une application de gestion d'une bibliothèque, elle peut identifier des acteurs comme « Bibliothécaire » et « Étudiant », ainsi que des fonctionnalités telles que « Emprunter un livre », « Retourner un livre » ou « Rechercher un livre ».

Elle peut également proposer des classes, leurs attributs et leurs méthodes, ou suggérer des relations entre les différentes classes. L'IA peut ainsi servir de partenaire de réflexion et permettre d'explorer rapidement plusieurs possibilités de conception.

Elle peut aussi transformer un modèle existant en une autre représentation. Par exemple, une description textuelle peut être transformée en code PlantUML permettant ensuite de générer un diagramme.

2.2 Importance d'un prompt bien cadré

La qualité du résultat fourni par une IA dépend fortement de la précision de la demande. Un prompt trop général peut produire un modèle incomplet, ambigu ou inadapté au contexte du projet.

Un bon prompt doit préciser, lorsque cela est nécessaire, le type de diagramme souhaité, le contexte du système, les acteurs, les fonctionnalités, les contraintes et le niveau de détail attendu. Par exemple, demander simplement « Fais-moi un diagramme UML pour une bibliothèque » laisse beaucoup trop de liberté à l'IA. Il est préférable de préciser les fonctionnalités du système, les utilisateurs concernés et le type de diagramme souhaité.

Il est également fréquent de devoir effectuer plusieurs itérations. Après une première proposition, l'utilisateur peut demander à l'IA de corriger une relation, d'ajouter une classe, de supprimer un élément inutile ou de respecter une contrainte particulière. Le modèle final est donc souvent obtenu progressivement.

2.3 Limites de l'IA dans la modélisation UML

Malgré ses capacités, l'IA possède certaines limites. Elle peut notamment mal comprendre le contexte métier ou interpréter de manière incorrecte une exigence ambiguë. Elle peut également proposer des éléments UML qui semblent cohérents mais qui ne correspondent pas réellement aux besoins du projet.

L'IA ne connaît pas nécessairement toutes les contraintes techniques, organisationnelles ou métier du système. Elle peut aussi simplifier excessivement un problème complexe ou, au contraire, ajouter des éléments inutiles.

Il faut également distinguer la validité syntaxique d'un diagramme de sa pertinence. Un diagramme peut être correctement écrit en UML tout en représentant une mauvaise conception du système.

2.4 Types d'erreurs possibles

L'IA peut produire différents types d'erreurs lors de la modélisation UML :

Table 2. Types d'erreurs possibles de l'IA en modélisation UML

Type d'erreur	Description
Erreurs d'interprétation	Mauvaise compréhension des besoins ou des exigences.
Erreurs de relations	Association incorrecte entre deux classes ou mauvaise utilisation de l'héritage, de la composition ou de l'agrégation.
Oublis	Acteurs, fonctionnalités, classes ou relations importantes absents.
Éléments inventés	Ajout de fonctionnalités ou de classes qui n'existent pas dans le cahier des charges.
Erreurs de notation	Utilisation incorrecte de certains éléments ou concepts UML.
Incohérences	Un diagramme de classes qui ne correspond pas, par exemple, aux fonctionnalités présentées dans un diagramme de cas d'utilisation.
Sur-modélisation	Modèle inutilement complexe, avec trop de classes ou de relations.

2.5 Importance de la validation humaine

La validation humaine reste donc indispensable. L'IA peut proposer rapidement des solutions, mais elle ne possède pas nécessairement une compréhension complète du contexte et des objectifs du projet.

Le développeur ou l'analyste doit vérifier que le modèle correspond réellement aux exigences, que les relations UML sont correctes et que le niveau de détail est adapté au projet. Il doit également comparer les différents diagrammes afin de détecter les incohérences.

L'utilisation de l'IA dans la modélisation UML doit ainsi être considérée comme une assistance à la conception et à la réflexion, et non comme un remplacement de l'expertise humaine. L'IA permet de gagner du temps et d'explorer différentes solutions, mais la décision finale et la validation du modèle doivent rester sous la responsabilité du concepteur.

> **À retenir**
>
> L'IA assiste la conception et accélère l'exploration des solutions ; la validation du modèle et la décision finale restent sous la responsabilité du concepteur.

3 Comparaison des outils UML

Table 3. Comparaison des outils UML

Outil	Prix	Facilité	Collaboration
StarUML	Payant	Facile	Non
Visual Paradigm CE	Gratuit (non commercial)	Moyenne	Limitée
draw.io	Gratuit	Très facile	Oui (via Google Drive)
Lucidchart	Gratuit limité / payant	Très facile	Oui, en temps réel
PowerDesigner (choix retenu)	**Payant**	**Moyenne**	**Limitée**

Notre choix

Nous avons choisi PowerDesigner car il s'agit d'un outil de modélisation professionnel et complet, qui prend en charge UML ainsi que d'autres types de modèles, et qui permet de produire des diagrammes structurés et cohérents pour l'ensemble du projet.

Partie II

Projet

Application de gestion d'un complexe sportif
Thématique : Sport & Event Management

1 Introduction

1.1 Contexte du projet

Les complexes sportifs proposent aujourd'hui plusieurs activités dans un même lieu : terrains de football, padel, tennis, basket, salle de musculation, cours collectifs ou piscine. Ils accueillent chaque jour un grand nombre de clients, d'abonnés, de coachs et d'équipes. Dans beaucoup de cas, la gestion se fait encore par téléphone, sur papier ou avec des fichiers Excel.

1.2 Problématique

Cette gestion manuelle pose plusieurs problèmes : réservations en double, difficulté à connaître la disponibilité des terrains en temps réel, suivi compliqué des abonnements et des paiements, et peu de visibilité pour le responsable sur l'activité du complexe.

> **Question centrale**
>
> Comment centraliser et simplifier la gestion d'un complexe sportif, pour le personnel comme pour les clients ?

1.3 Motivation du projet

Nous avons choisi ce thème car il est proche de la vie quotidienne et facile à comprendre. Il est aussi très flexible : on peut commencer par les fonctionnalités de base (réservation, abonnements) puis ajouter d'autres modules (tournois, coachs, statistiques). C'est donc un bon sujet pour appliquer les différents diagrammes UML.

1.4 Objectifs du projet

- Permettre aux clients de réserver un terrain ou une séance en ligne.
- Gérer les abonnements et les paiements.
- Organiser les plannings des terrains, des cours et des coachs.
- Gérer des événements et des tournois.
- Donner au responsable un tableau de bord sur l'activité du complexe.

1.5 Périmètre du projet

Le projet concerne un seul complexe sportif avec plusieurs installations et plusieurs sports. L'application est conçue de façon modulaire pour pouvoir ajouter de nouveaux sports ou services plus tard.

Table 4. Périmètre du projet

Périmètre	Éléments
Inclus	Réservations, abonnements, plannings, paiements et événements.
Exclus	Comptabilité complète et gestion des salaires du personnel.

2 Présentation générale de la solution

2.1 Description de l'application

La solution est une application (web et mobile) qui regroupe toute la gestion du complexe au même endroit. Chaque utilisateur a un espace selon son rôle : le client réserve et paie, le coach consulte son planning, le réceptionniste gère l'accueil et le responsable suit l'activité.

2.2 Principe de fonctionnement

1. Le client crée un compte et se connecte.
2. Il consulte les installations disponibles (sport, date, heure).
3. Il réserve un créneau ou s'inscrit à un cours ou un tournoi.
4. Il paie en ligne ou sur place, et reçoit une confirmation.
5. Le personnel voit les réservations en temps réel et gère l'accueil.
6. Le responsable consulte les statistiques (taux d'occupation, revenus).

2.3 Valeur ajoutée de la solution

- Fin des réservations en double grâce à un planning en temps réel.
- Réservation possible à tout moment, sans appeler le complexe.
- Gain de temps pour le personnel.
- Meilleur suivi des abonnements et des paiements.
- Solution évolutive : nouveaux sports ou modules faciles à ajouter.

2.4 Utilisateurs et acteurs du système

Table 5. Utilisateurs du système et leurs responsabilités

Utilisateur	Responsabilités
Client / Visiteur	Consulte les disponibilités des terrains et des salles ; réserve, modifie ou annule un créneau ; paie sa réservation ; s'inscrit à un événement ou un tournoi.
Abonné	Souscrit et renouvelle son abonnement ; accède aux cours et aux avantages de son abonnement.
Coach	Consulte son planning ; gère ses cours et ses participants.
Réceptionniste	Gère l'accueil des clients ; enregistre les réservations et les paiements sur place.
Administrateur	Gère les installations, les horaires et les tarifs ; gère les utilisateurs et le personnel ; consulte les statistiques du complexe.

3 Étude des besoins

Du point de vue du client (le responsable du complexe sportif), l'application doit permettre de :

- centraliser toute la gestion du complexe dans une seule application ;
- permettre aux clients de réserver un terrain ou une séance en ligne, à tout moment ;
- éviter les réservations en double avec un planning à jour en temps réel ;
- gérer les abonnements des membres et suivre leurs paiements ;
- organiser les cours et les plannings des coachs ;
- créer et gérer des événements et des tournois ;
- envoyer des confirmations et des rappels aux clients ;
- suivre l'activité du complexe (occupation des terrains, revenus) ;
- disposer d'une application simple à utiliser, sur ordinateur et sur téléphone ;
- protéger les données des clients et des paiements ;
- pouvoir ajouter facilement de nouveaux sports ou services plus tard.

4 Diagramme de cas d'utilisation global

Le diagramme ci-dessous présente une vue d'ensemble des fonctionnalités du système de gestion du complexe sportif et des acteurs qui les utilisent.

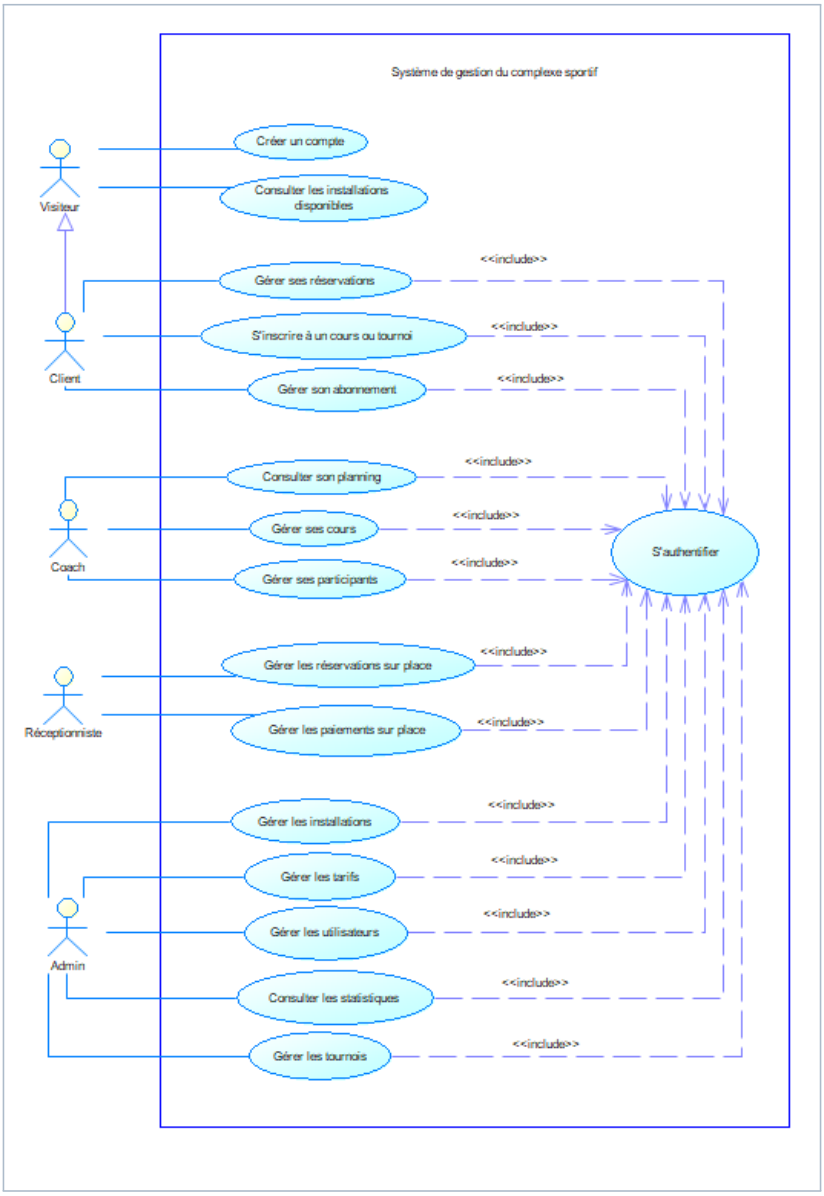


Figure 1. Diagramme de cas d'utilisation global du système de gestion du complexe sportif

Description

Le système est utilisé par cinq acteurs. Le **Client** hérite du **Visiteur** : il dispose donc de toutes ses possibilités, auxquelles s'ajoutent celles liées à son compte. La plupart des cas d'utilisation nécessitent une connexion : ils sont reliés par une relation `«include»` au cas « S'authentifier ».

Table 6. Acteurs et cas d'utilisation du système

Acteur	Cas d'utilisation
Visiteur	Créer un compte ; consulter les installations disponibles.
Client	Gérer ses réservations ; s'inscrire à un cours ou un tournoi ; gérer son abonnement.
Coach	Consulter son planning ; gérer ses cours ; gérer ses participants.
Réceptionniste	Gérer les réservations sur place ; gérer les paiements sur place.
Admin	Gérer les installations, les tarifs et les utilisateurs ; consulter les statistiques ; gérer les tournois.

5 Diagramme de cas d'utilisation : Authentification

Le cas « S'authentifier », inclus par la plupart des fonctionnalités du diagramme global, est détaillé ci-dessous. Il met en place une authentification à deux facteurs (2FA).

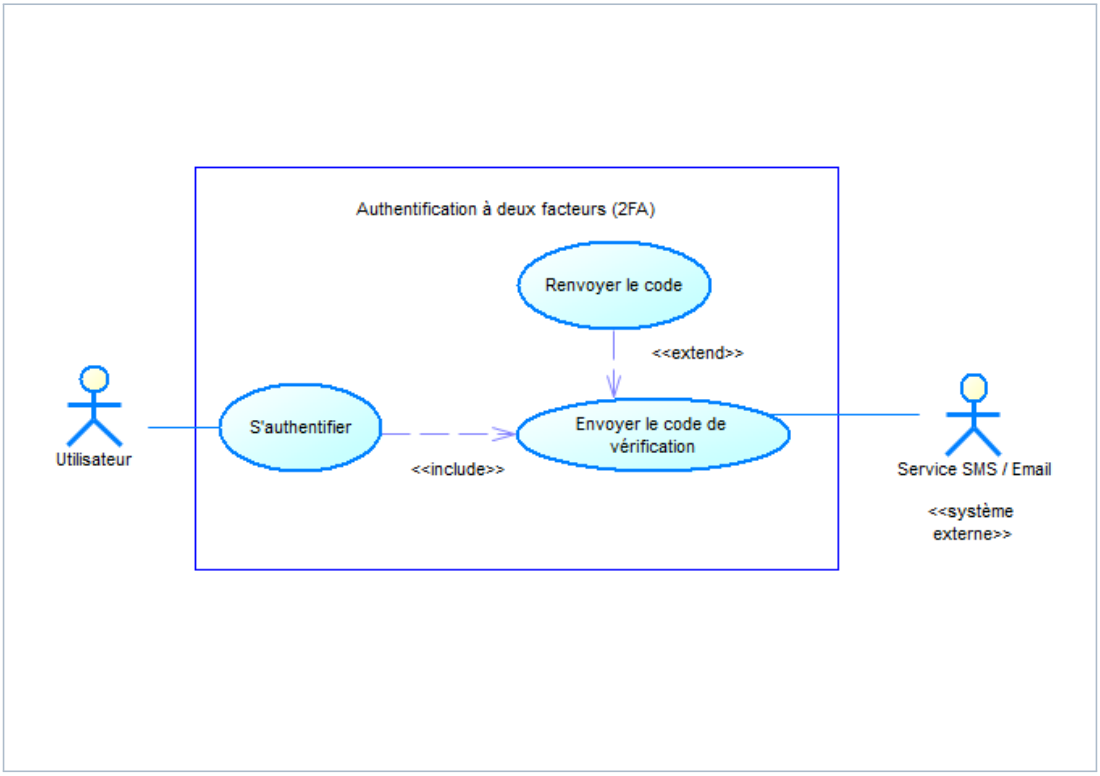


Figure 2. Diagramme de cas d'utilisation : authentification à deux facteurs (2FA)

Description

L'**Utilisateur** lance le cas « S'authentifier ». Celui-ci **inclut** (`«include»`) systématiquement le cas « Envoyer le code de vérification », assuré par le **Service SMS / Email**, un système externe. Si l'utilisateur ne reçoit pas le code, le cas « Renvoyer le code » **étend** (`«extend»`) l'envoi du code de vérification.

Table 7. Éléments du diagramme d'authentification

Élément	Rôle
Utilisateur	Acteur principal qui souhaite se connecter au système.
Service SMS / Email	Système externe qui envoie le code de vérification.
S'authentifier	Cas principal de connexion de l'utilisateur.
Envoyer le code de vérification	Cas toujours exécuté lors de l'authentification (`«include»`).
Renvoyer le code	Cas optionnel, exécuté si le code n'est pas reçu (`«extend»`).

6 Diagrammes de cas d'utilisation détaillés : Client

Cette section détaille les trois principales fonctionnalités de l'acteur **Client** présentes dans le diagramme global.

6.1 Gérer ses réservations

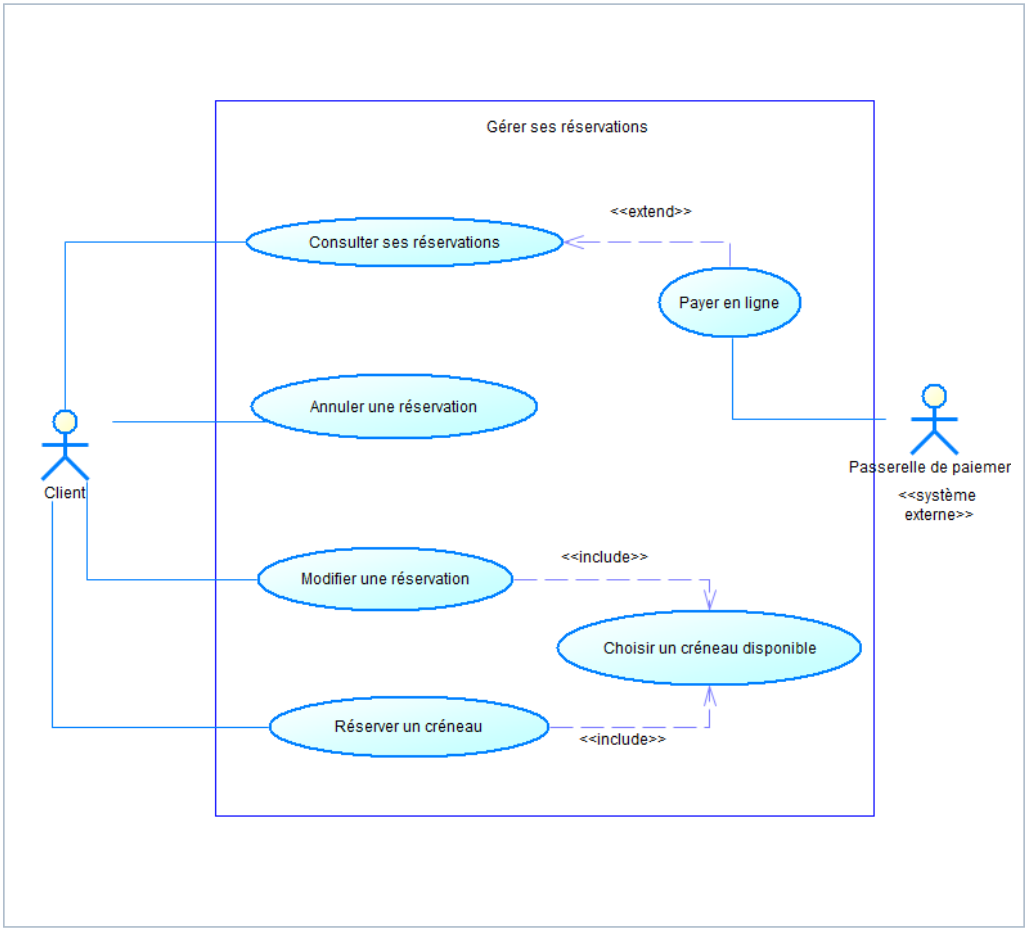


Figure 3. Diagramme de cas d'utilisation : gérer ses réservations

Le **Client** peut consulter ses réservations, annuler une réservation, modifier une réservation et réserver un créneau. Les cas « Modifier une réservation » et « Réserver un créneau » **incluent** (`«include»`) le cas « Choisir un créneau disponible ». Le cas « Payer en ligne », assuré par la **passerelle de paiement** (système externe), **étend** (`«extend»`) le cas « Consulter ses réservations ».

6.2 S'inscrire à un cours ou tournoi

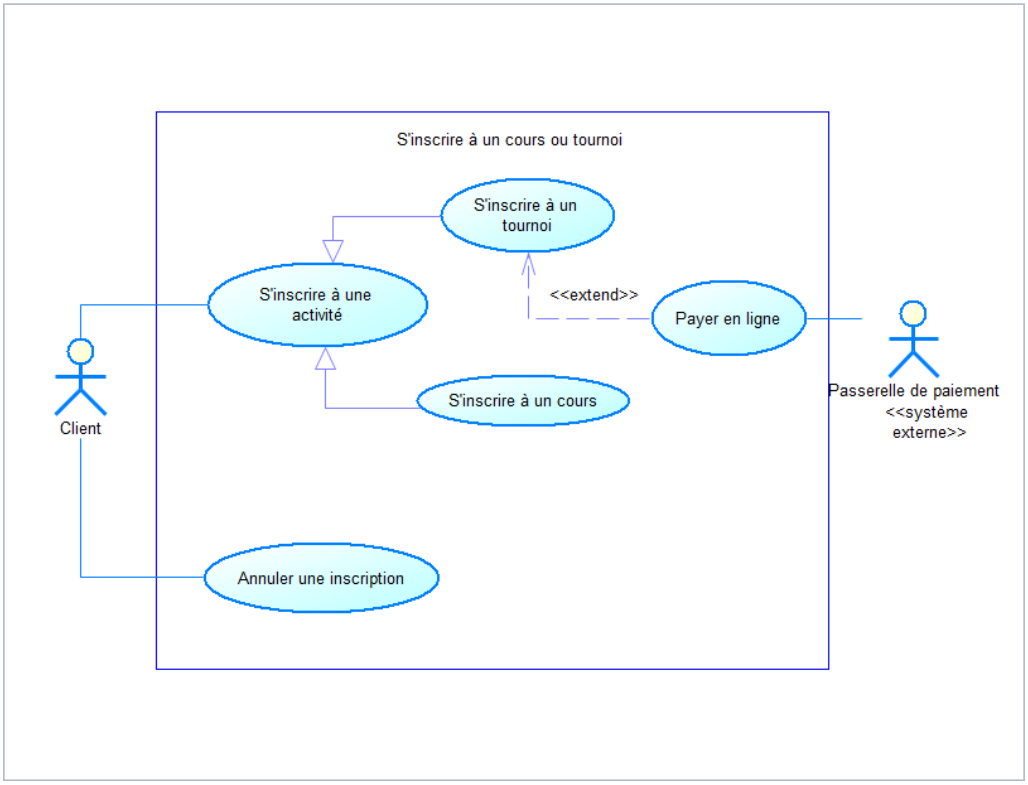


Figure 4. Diagramme de cas d'utilisation : s'inscrire à un cours ou tournoi

Le **Client** peut s'inscrire à une activité ou annuler une inscription. Le cas « S'inscrire à une activité » est généralisé en deux cas plus précis : « S'inscrire à un cours » et « S'inscrire à un tournoi ». Le cas « Payer en ligne », assuré par la **passerelle de paiement** (système externe), **étend** (`«extend»`) l'inscription à un tournoi.

6.3 Gérer son abonnement

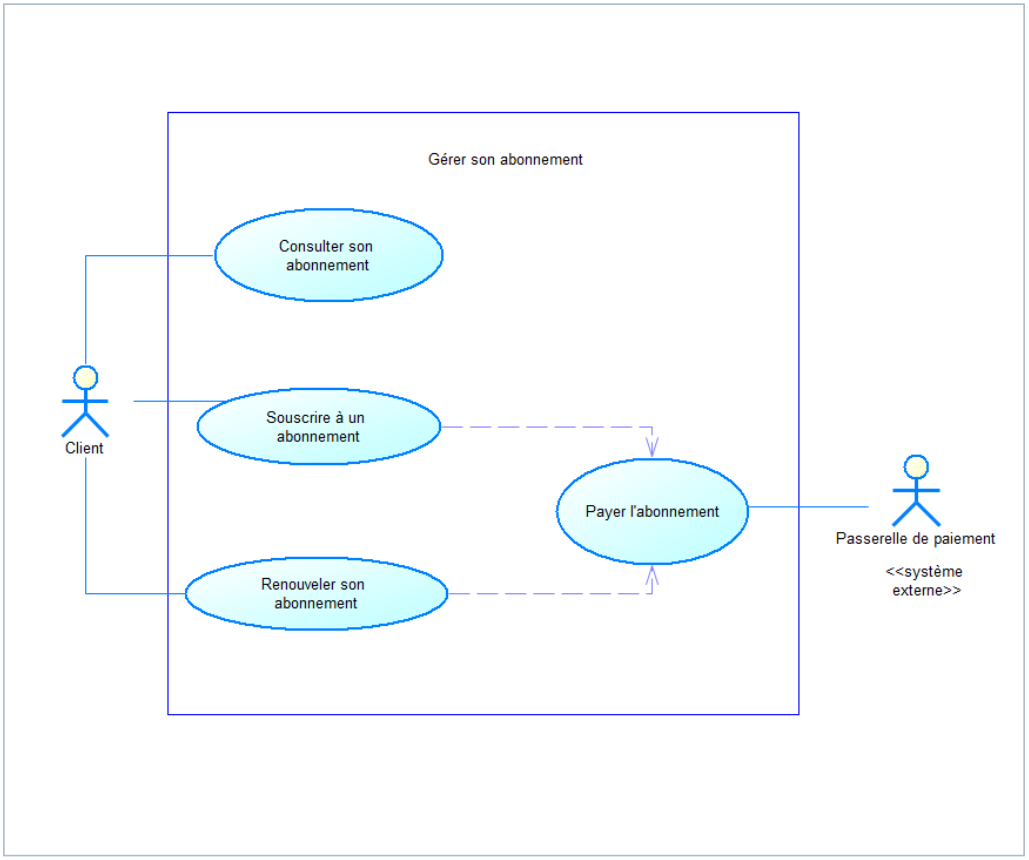


Figure 5. Diagramme de cas d'utilisation : gérer son abonnement

Le **Client** peut consulter son abonnement, souscrire à un abonnement ou le renouveler. Les cas « Souscrire à un abonnement » et « Renouveler son abonnement » dépendent du cas « Payer l'abonnement », assuré par la **passerelle de paiement** (système externe).

7 Diagrammes de cas d'utilisation détaillés : Coach

Cette section détaille les fonctionnalités de l'acteur **Coach** présentes dans le diagramme global.

7.1 Gérer ses cours

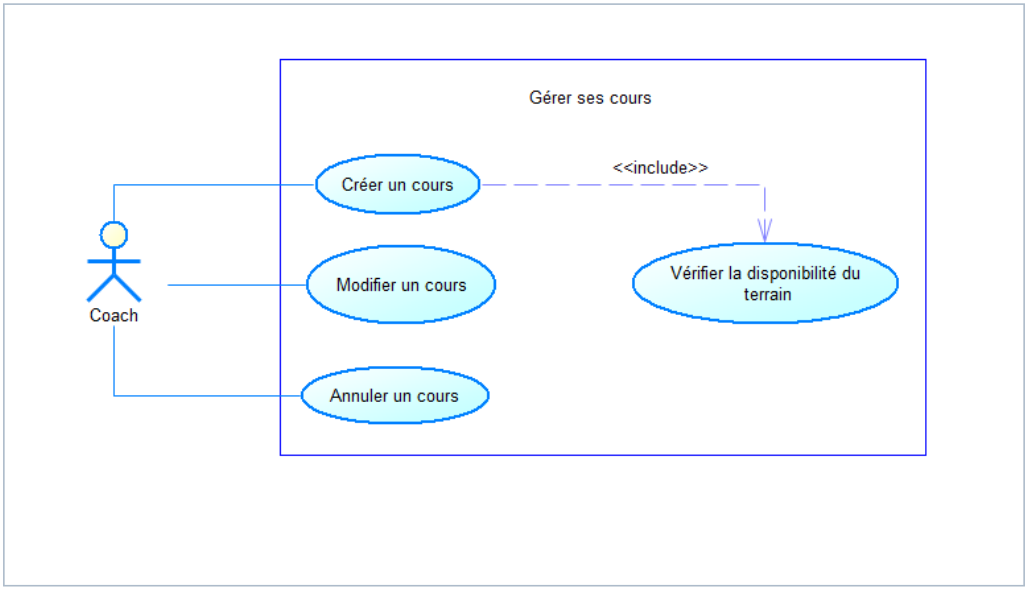


Figure 6. Diagramme de cas d'utilisation : gérer ses cours

Le **Coach** peut créer un cours, modifier un cours et annuler un cours. Le cas « Créer un cours » **inclut** (`«include»`) le cas « Vérifier la disponibilité du terrain », afin d'éviter les réservations en double.

7.2 Gérer ses participants

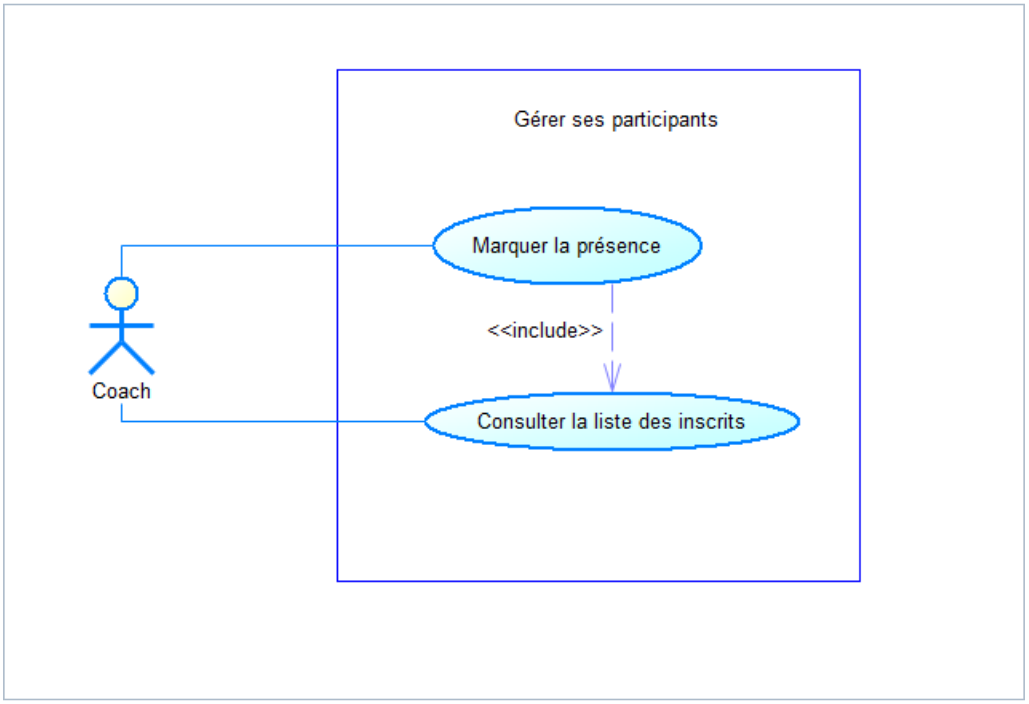


Figure 7. Diagramme de cas d'utilisation : gérer ses participants

Le **Coach** peut marquer la présence des participants et consulter la liste des inscrits. Le cas « Marquer la présence » **inclut** (`«include»`) le cas « Consulter la liste des inscrits ».

8 Diagrammes de cas d'utilisation détaillés : Admin

Cette section détaille les fonctionnalités de l'acteur **Admin** présentes dans le diagramme global.

8.1 Gérer les installations

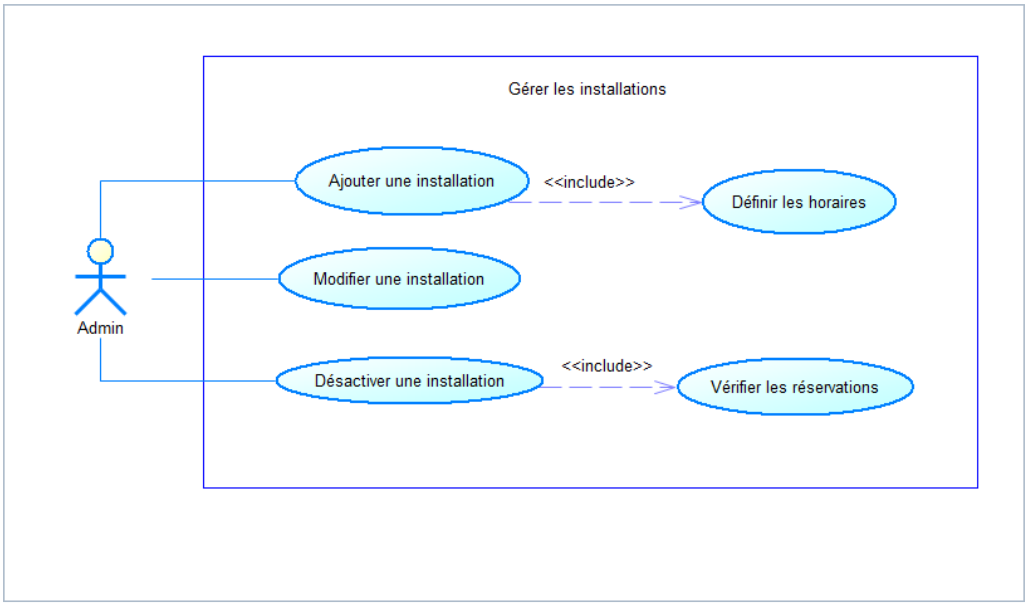


Figure 8. Diagramme de cas d'utilisation : gérer les installations

L'**Admin** peut ajouter, modifier et désactiver une installation. Le cas « Ajouter une installation » **inclut** (`«include»`) le cas « Définir les horaires ». Le cas « Désactiver une installation » **inclut** (`«include»`) le cas « Vérifier les réservations ».

8.2 Gérer les tarifs

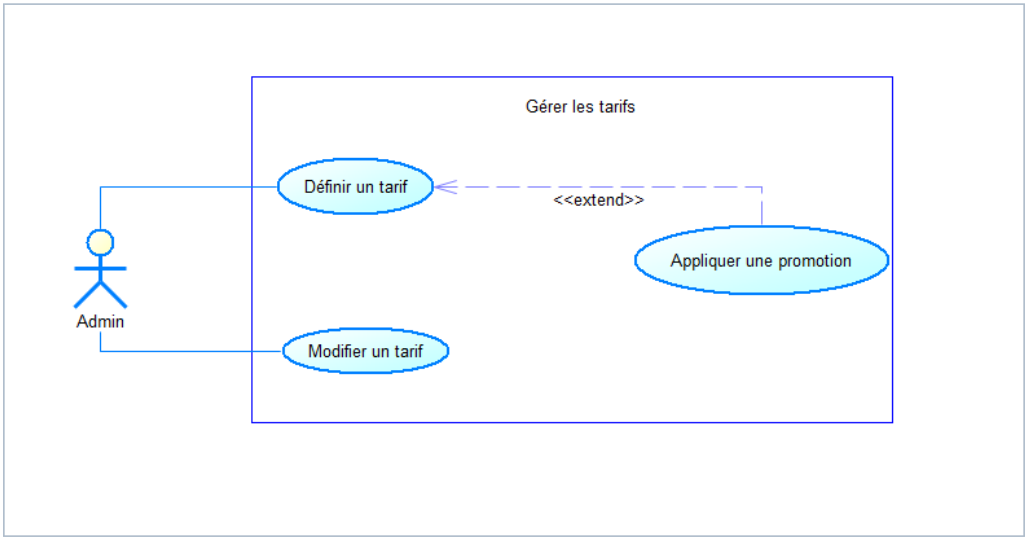


Figure 9. Diagramme de cas d'utilisation : gérer les tarifs

L'**Admin** peut définir un tarif et modifier un tarif. Le cas « Appliquer une promotion » **étend** (`«extend»`) le cas « Définir un tarif ».

8.3 Gérer les utilisateurs

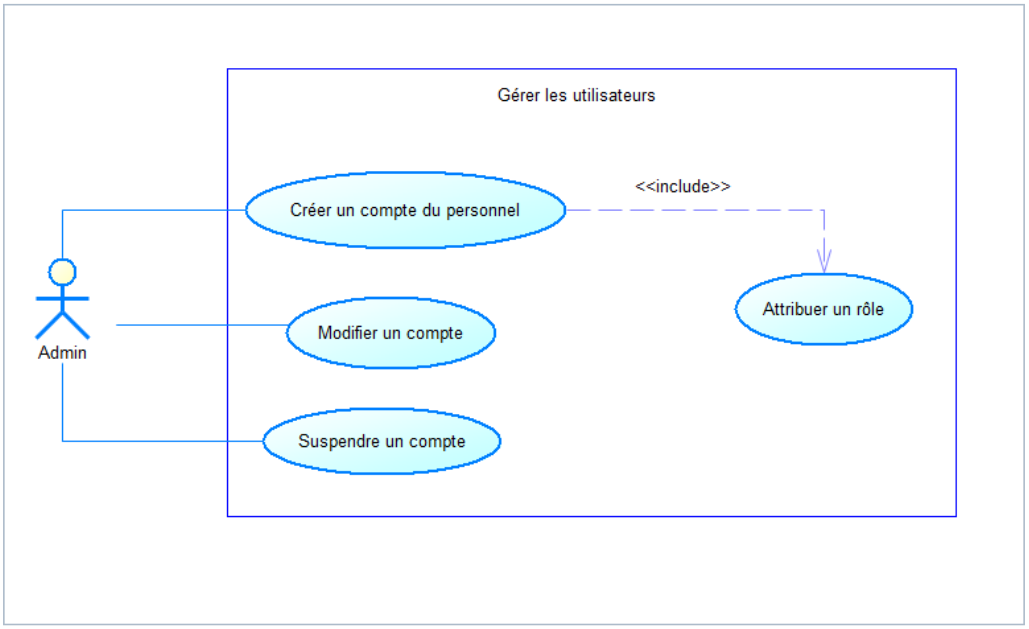


Figure 10. Diagramme de cas d'utilisation : gérer les utilisateurs

L'**Admin** peut créer un compte du personnel, modifier un compte et suspendre un compte. Le cas « Créer un compte du personnel » **inclut** (`«include»`) le cas « Attribuer un rôle ».

8.4 Gérer les tournois

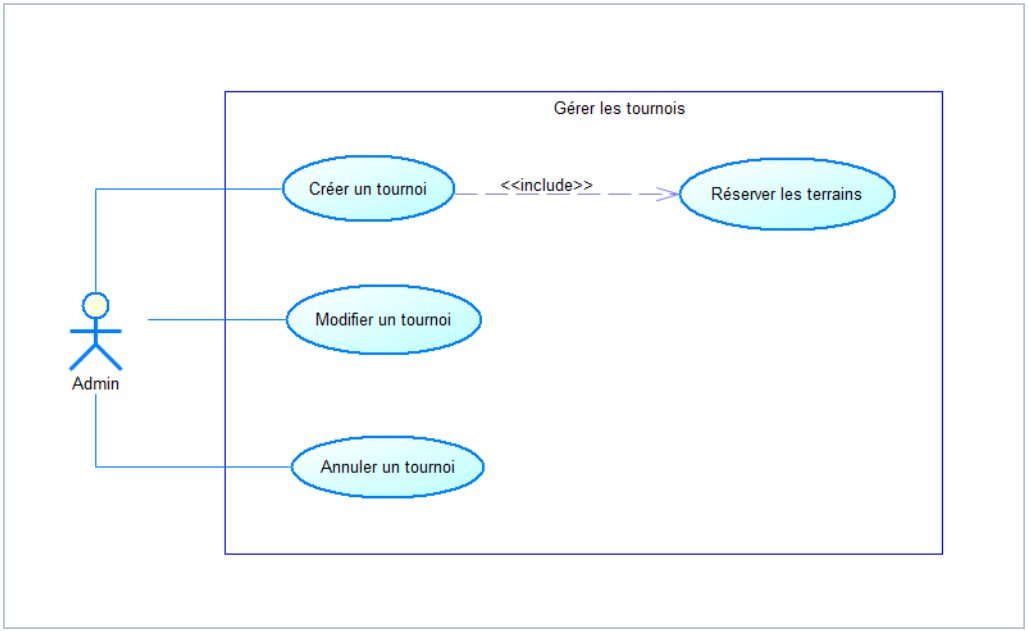


Figure 11. Diagramme de cas d'utilisation : gérer les tournois

L'**Admin** peut créer, modifier et annuler un tournoi. Le cas « Créer un tournoi » **inclut** (`«include»`) le cas « Réserver les terrains ».

9 Diagrammes de cas d'utilisation détaillés : Réceptionniste

Cette section détaille les fonctionnalités de l'acteur **Réceptionniste** présentes dans le diagramme global.

9.1 Gérer les réservations sur place

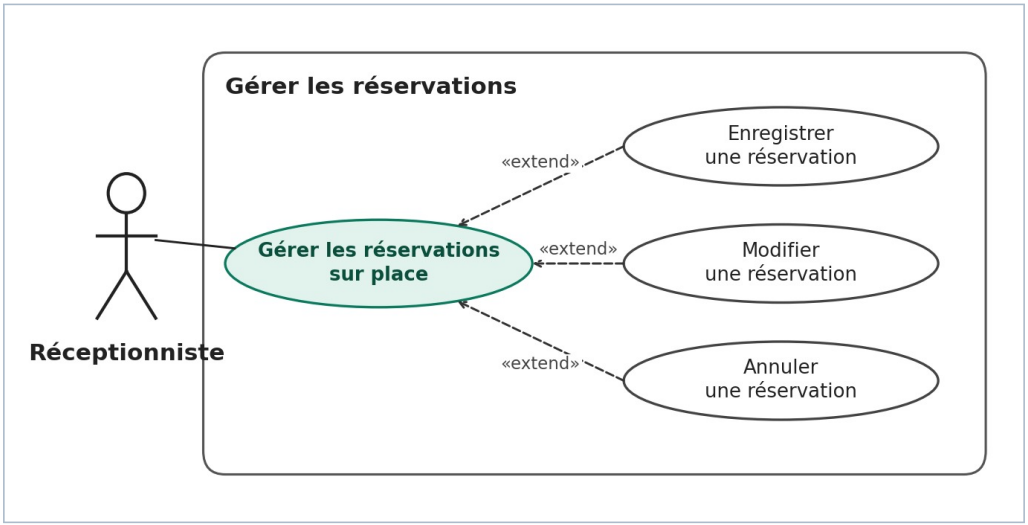


Figure 12. Diagramme de cas d'utilisation : gérer les réservations sur place

Le **Réceptionniste** gère les réservations sur place. Les cas « Enregistrer une réservation », « Modifier une réservation » et « Annuler une réservation » **étendent** (`«extend»`) le cas « Gérer les réservations sur place ».

9.2 Gérer les paiements sur place

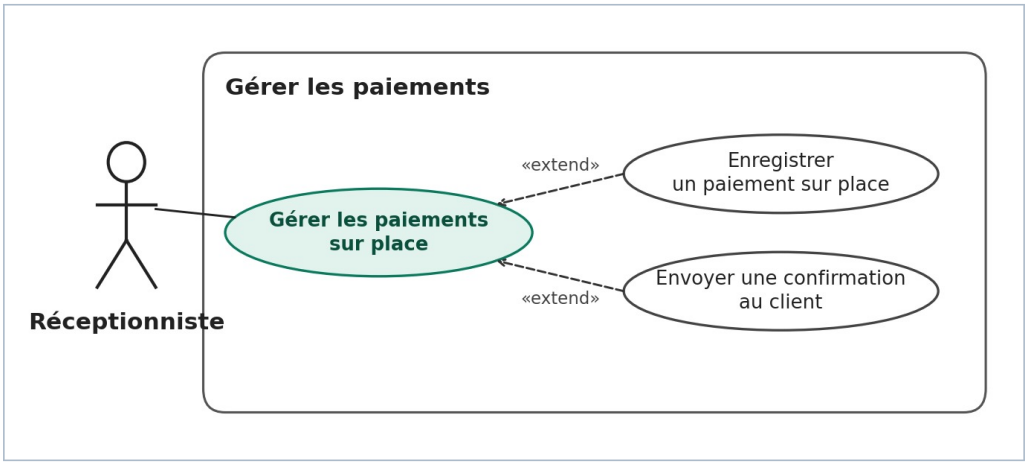


Figure 13. Diagramme de cas d'utilisation : gérer les paiements sur place

Le **Réceptionniste** gère les paiements sur place. Les cas « Enregistrer un paiement sur place » et « Envoyer une confirmation au client » **étendent** (`«extend»`) le cas « Gérer les paiements sur place ».

10 Diagramme de classes

Le diagramme de classes décrit la structure statique du système : les classes, leurs attributs, leurs opérations et les relations qui les lient. Il est organisé en six paquetages, résumés dans le tableau ci-dessous.

Table 8. Organisation du diagramme de classes en paquetages

Paquetage	Classes	Rôle
Utilisateurs	`Utilisateur` (abstraite), `Client`, `Coach`, `Receptionniste`, `Administrateur`	Acteurs du système et authentification
Activités	`Activite` (abstraite), `Cours`, `Tournoi`, `Inscription`	Séances collectives, tournois et participation des clients
Installations et réservations	`Installation`, `Horaire`, `Creneau`, `Reservation`	Terrains, salles, piscine, disponibilités et réservations
Abonnements et tarifs	`Abonnement`, `Tarif`, `Promotion`	Formules d'abonnement, prix et promotions
Paiements et communication	`Paiement`, `Notification`, `CodeVerification`	Règlements, messages envoyés et code de l'authentification à deux facteurs
Énumérations	`StatutCompte`, `TypeInstallation`, `StatutCreneau`, `StatutReservation`, `CanalReservation`, `StatutActivite`, `StatutInscription`, `StatutAbonnement`, `ModePaiement`, `StatutPaiement`, `CanalEnvoi`, `TypeNotification`	Types et statuts utilisés par les classes

10.1 Diagramme de classes global

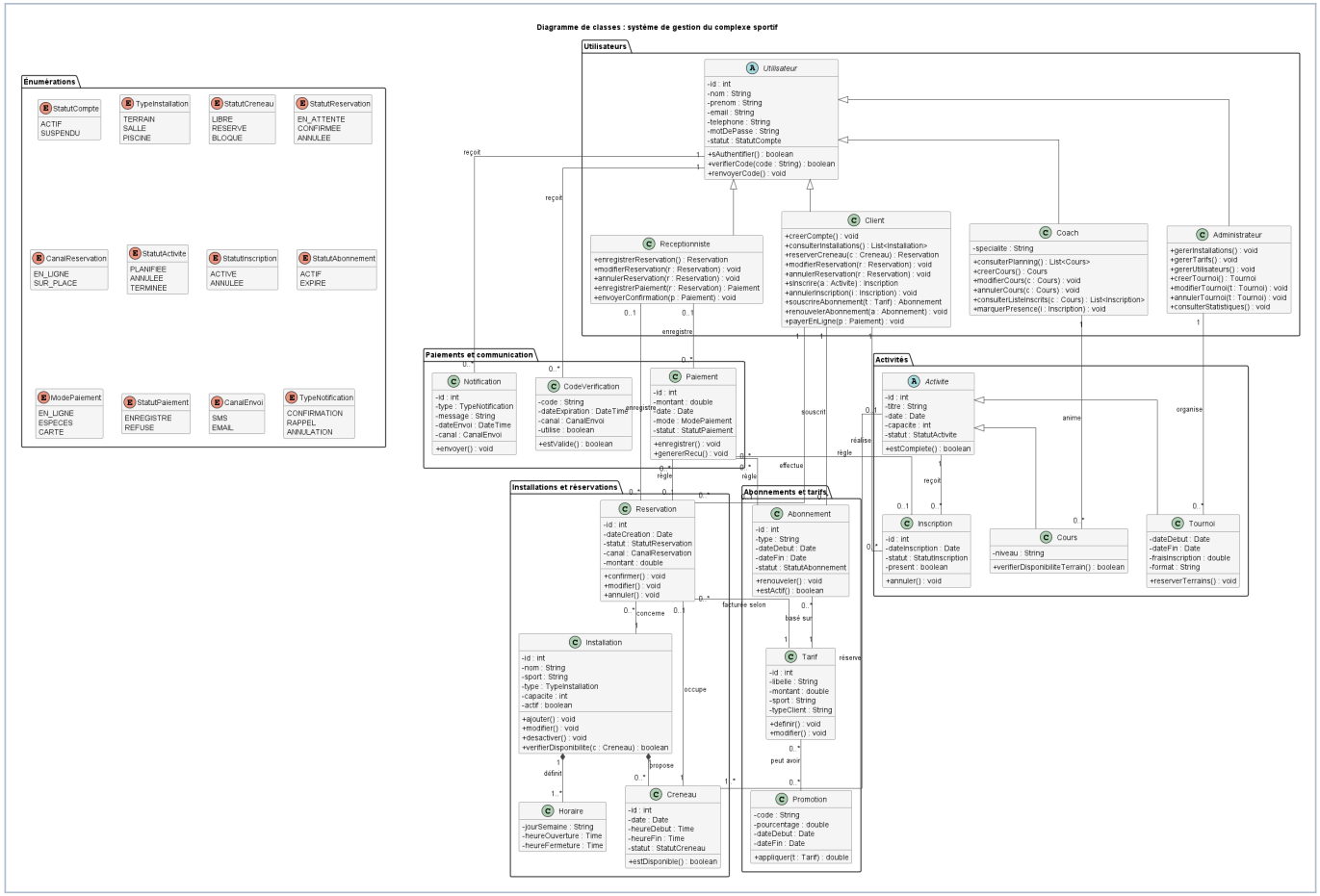


Figure 14. Diagramme de classes global du système de gestion du complexe sportif

Description

- **Utilisateurs** : la classe abstraite `Utilisateur` (id, nom, prénom, email, téléphone, mot de passe, statut) est spécialisée en `Client`, `Coach`, `Receptionniste` et `Administrateur`. Elle porte aussi les opérations d'authentification à deux facteurs (`sAuthentifier`, `verifierCode`, `renvoyerCode`).
- **Activités** : la classe abstraite `Activite` est spécialisée en `Cours` (animé par un coach) et `Tournoi` (organisé par un administrateur). Un client y participe via une `Inscription`.
- **Installations et réservations** : une `Installation` est composée d'`Horaires` et de `Creneaux`. Une `Reservation` concerne une installation et occupe un créneau.
- **Abonnements et tarifs** : un `Abonnement` est souscrit par un client et basé sur un `Tarif`, qui peut avoir des `Promotions`.
- **Paiements et communication** : un `Paiement` règle une réservation, un abonnement ou une inscription. Les `Notifications` et les `CodeVerification` sont reçus par l'utilisateur.
- **Énumérations** : elles définissent les statuts et les types utilisés par les classes (`StatutCompte`, `StatutReservation`, `ModePaiement`, `CanalEnvoi`, etc.).

Le tableau suivant récapitule les relations du diagramme. Les valeurs entre parenthèses sont les multiplicités indiquées sur le diagramme, du côté de la classe concernée.

10.2 Détail des classes

Les figures suivantes détaillent chaque classe avec ses relations directes.

Paquetage Utilisateurs

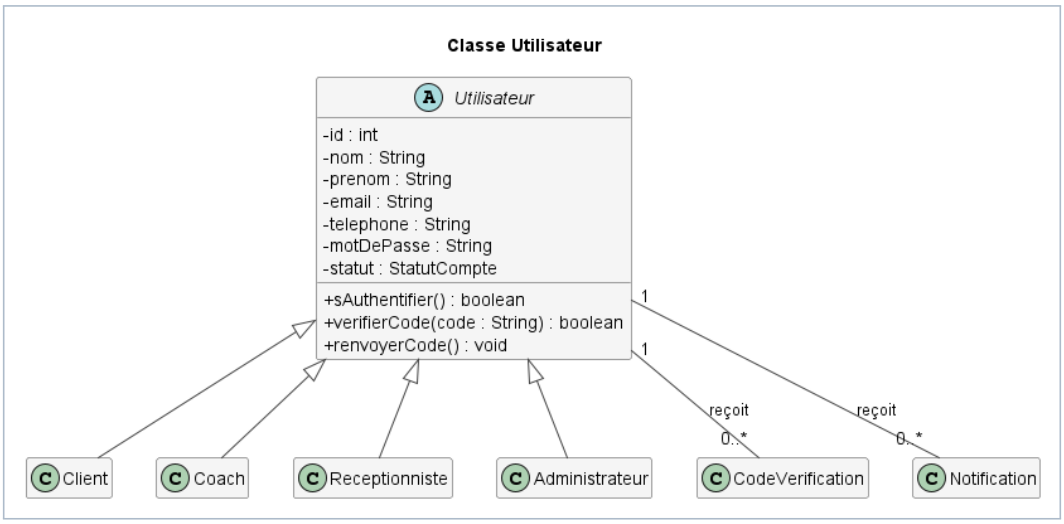


Figure 15. Classe abstraite `Utilisateur`

Table 9. Principales relations du diagramme de classes

Relation	Classes reliées (multiplicité)	Nature
Spécialisation	`Utilisateur` ← `Client`, `Coach`, `Receptionniste`, `Administrateur`	**Généralisation**
Spécialisation	`Activite` ← `Cours`, `Tournoi`	**Généralisation**
effectue	`Client` (1) – `Reservation` (0..*)	**Association**
réalise	`Client` (1) – `Inscription` (0..*)	**Association**
souscrit	`Client` (1) – `Abonnement` (0..*)	**Association**
anime	`Coach` (1) – `Cours` (0..*)	**Association**
organise	`Administrateur` (1) – `Tournoi` (0..*)	**Association**
enregistre	`Receptionniste` (0..1) – `Reservation` (0..*) ; `Receptionniste` (0..1) – `Paiement` (0..*)	**Association**
concerne	`Reservation` (0..*) – `Installation` (1)	**Association**
occupe	`Reservation` (0..1) – `Creneau` (1)	**Association**
facturée selon	`Reservation` (0..*) – `Tarif` (1)	**Association**
basé sur	`Abonnement` (0..*) – `Tarif` (1)	**Association**
règle	`Paiement` (0..*) – `Reservation`, `Abonnement`, `Inscription` (0..1)	**Association**
reçoit	`Activite` (1) – `Inscription` (0..*)	**Association**
réserve	`Activite` (0..1) – `Creneau` (1..*)	**Association**
propose	`Installation` (1) – `Creneau` (0..*)	**Composition**
définit	`Installation` (1) – `Horaire` (1..*)	**Composition**
peut avoir	`Tarif` (0..*) – `Promotion` (0..*)	**Association**
reçoit	`Utilisateur` (1) – `CodeVerification` (0..*) ; `Utilisateur` (1) – `Notification` (0..*)	**Association**

La classe abstraite `Utilisateur` regroupe les attributs communs (id, nom, prénom, email, téléphone, mot de passe, statut) et les opérations d'authentification. Elle est spécialisée en `Client`, `Coach`, `Receptionniste` et `Administrateur`, et reçoit des codes de vérification et des notifications (multiplicité $1 \to 0..*$).

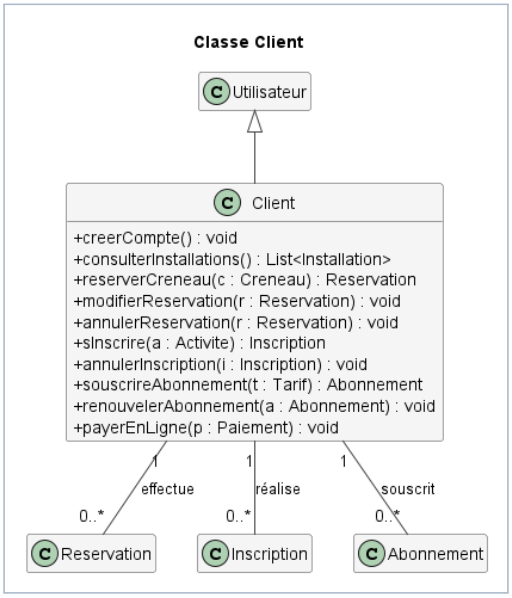


Figure 16. Classe `Client`

Le `Client` hérite d'`Utilisateur`. Il effectue des réservations, réalise des inscriptions et souscrit des abonnements (chaque relation est de multiplicité $1 \to 0..*$).

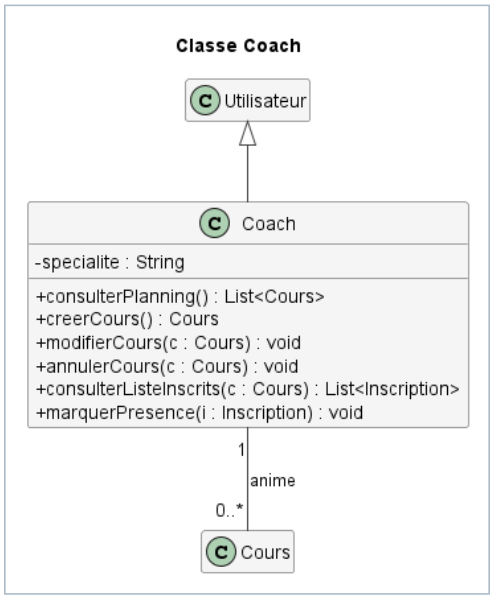


Figure 17. Classe `Coach`

Le `Coach` possède un attribut `specialite` et anime zéro à plusieurs cours. Il peut consulter son planning, gérer ses cours et marquer la présence des inscrits.

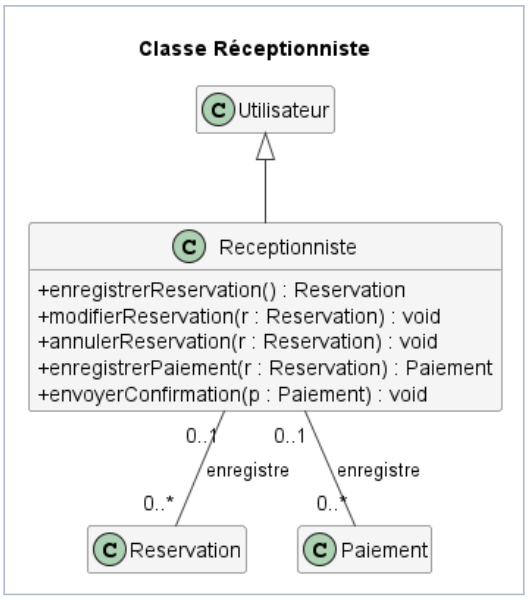


Figure 18. Classe `Receptionniste`

Le `Receptionniste` enregistre les réservations et les paiements sur place (multiplicité $0..1 \rightarrow 0..*$) et peut envoyer une confirmation au client.

Paquetage Activités

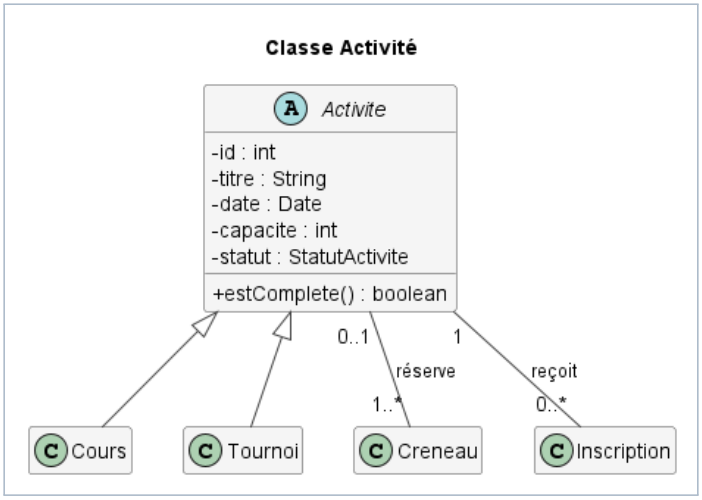


Figure 19. Classe abstraite `Activite`

`Activite` est une classe abstraite généralisée par `Cours` et `Tournoi`. Elle réserve un ou plusieurs créneaux et reçoit des inscriptions.

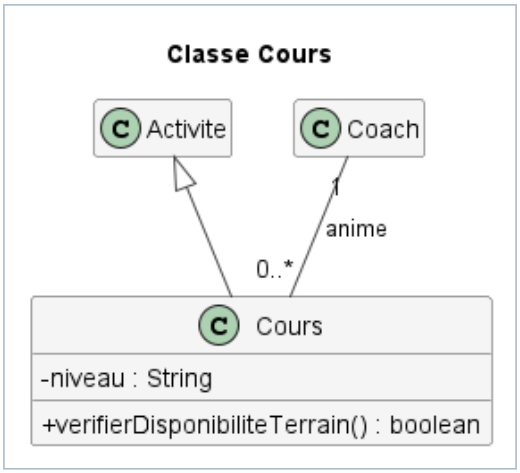


Figure 20. Classe `Cours`

Le `Cours` hérite d'`Activite`, possède un niveau et peut vérifier la disponibilité du terrain avant sa création.

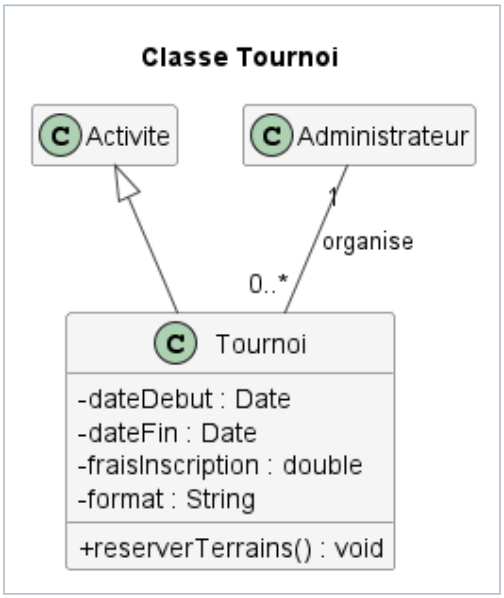


Figure 21. Classe `Tournoi`

Le `Tournoi` hérite d'`Activite`. Il possède des dates de début et de fin, des frais d'inscription et un format, et il est organisé par un administrateur (multiplicité $1 \rightarrow 0..*$). Il peut réserver les terrains nécessaires.

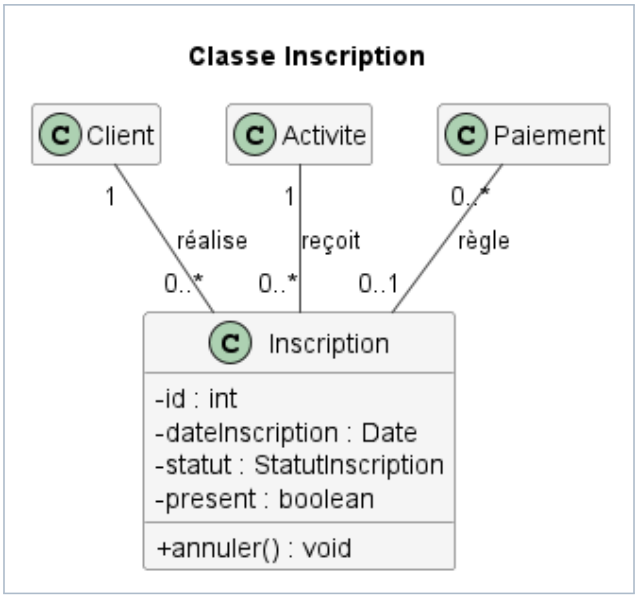


Figure 22. Classe `Inscription`

L'`Inscription` relie un client à une activité. Elle peut être réglée par un paiement (multiplicité 0..1) et porte l'attribut `present` utilisé pour marquer la présence.

Paquetage Installations et réservations

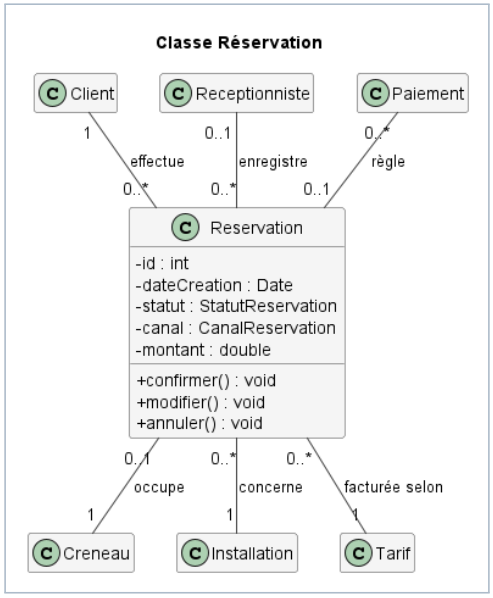


Figure 23. Classe `Reservation`

La `Reservation` est effectuée par un client, peut être enregistrée par un réceptionniste et réglée par un paiement. Elle occupe un créneau, concerne une installation et est facturée selon un tarif. Elle peut être confirmée, modifiée ou annulée.

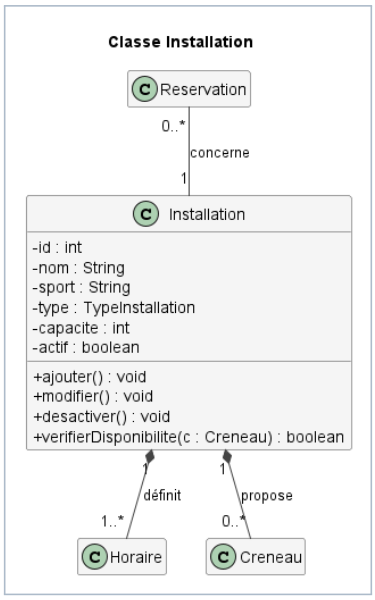


Figure 24. Classe `Installation`

L'`Installation` (terrain, salle ou piscine) est composée d'horaires et de créneaux, et peut être concernée par plusieurs réservations.

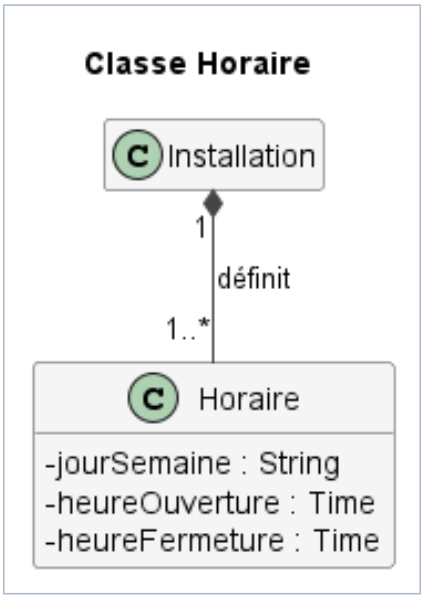


Figure 25. Classe `Horaire`

Un `Horaire` définit le jour de la semaine ainsi que les heures d'ouverture et de fermeture. Chaque installation définit au moins un horaire (composition).

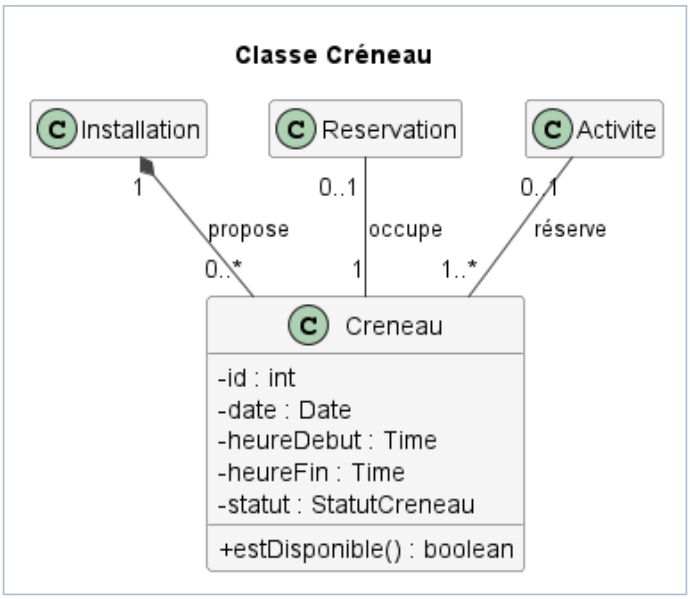


Figure 26. Classe `Creneau`

Le `Creneau` est proposé par une installation (composition). Il est occupé par au plus une réservation et peut être réservé par une activité.

Paquetage Abonnements et tarifs

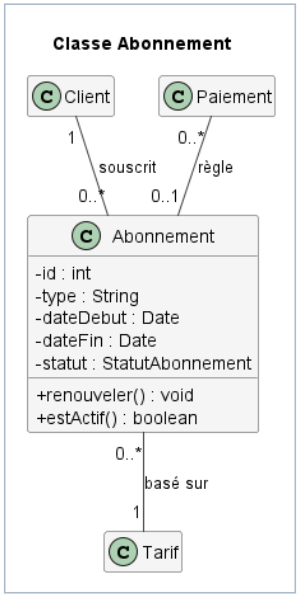


Figure 27. Classe `Abonnement`

L'`Abonnement` est souscrit par un client, basé sur un tarif et réglé par un paiement. Il peut être renouvelé et son état peut être vérifié (`estActif`).

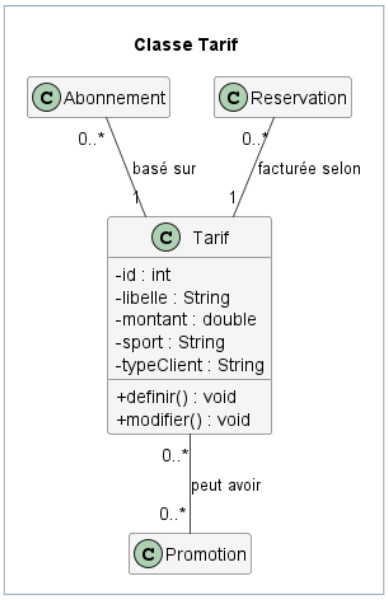


Figure 28. Classe `Tarif`

Le `Tarif` (libellé, montant, sport, type de client) sert de base aux abonnements et à la facturation des réservations. Il peut avoir des promotions.

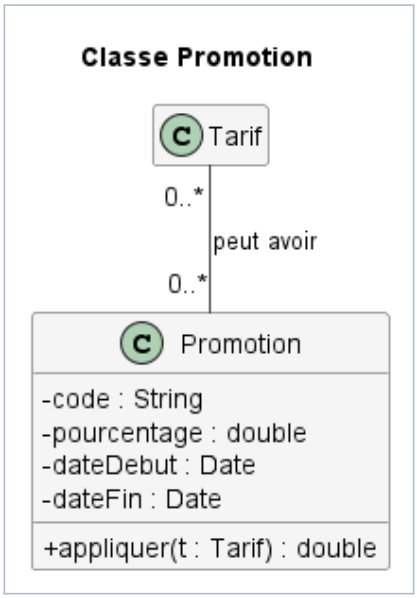


Figure 29. Classe `Promotion`

Une `Promotion` (code, pourcentage, période de validité) s'applique à un ou plusieurs tarifs.

Paquetage Paiements et communication

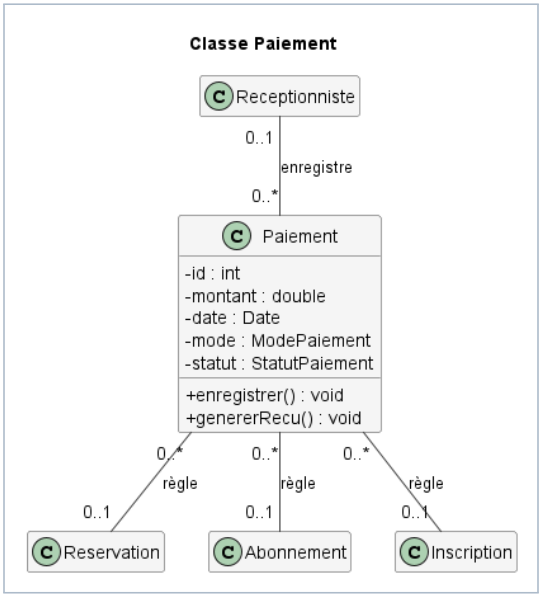


Figure 30. Classe `Paiement`

Le `Paiement` règle une réservation, un abonnement ou une inscription. Il est enregistré par un réceptionniste et peut générer un reçu.

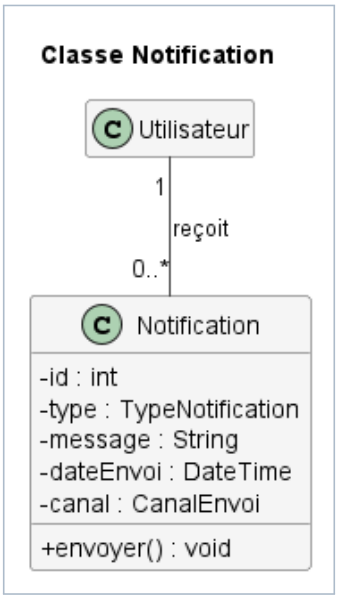


Figure 31. Classe `Notification`

Une `Notification` (confirmation, rappel ou annulation) est envoyée par SMS ou email à l'utilisateur.

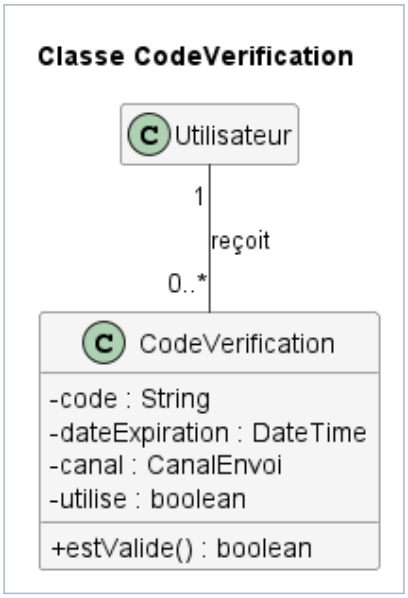


Figure 32. Classe `CodeVerification`

Le `CodeVerification` est le code temporaire utilisé par l'authentification à deux facteurs. Il possède une date d'expiration et un indicateur d'utilisation.

11 Diagrammes de séquence

Les diagrammes de séquence décrivent le déroulement dans le temps des échanges entre les acteurs et les composants du système. Ils suivent l'architecture *Interface / Contrôleur / Base de données*.

Table 10. Récapitulatif des diagrammes de séquence

Scénario	Participants	Fragments
Authentification à deux facteurs	Utilisateur, Interface, AuthController, Base de données, Service SMS / Email (externe)	`alt` (imbriqué), `opt`
Créer un tournoi	Admin, Interface, TournoiController, Base de données	`alt`, `loop`
Gérer les paiements sur place	Réceptionniste, Interface, PaiementController, Base de données	`alt`, `opt`

11.1 Authentification à deux facteurs

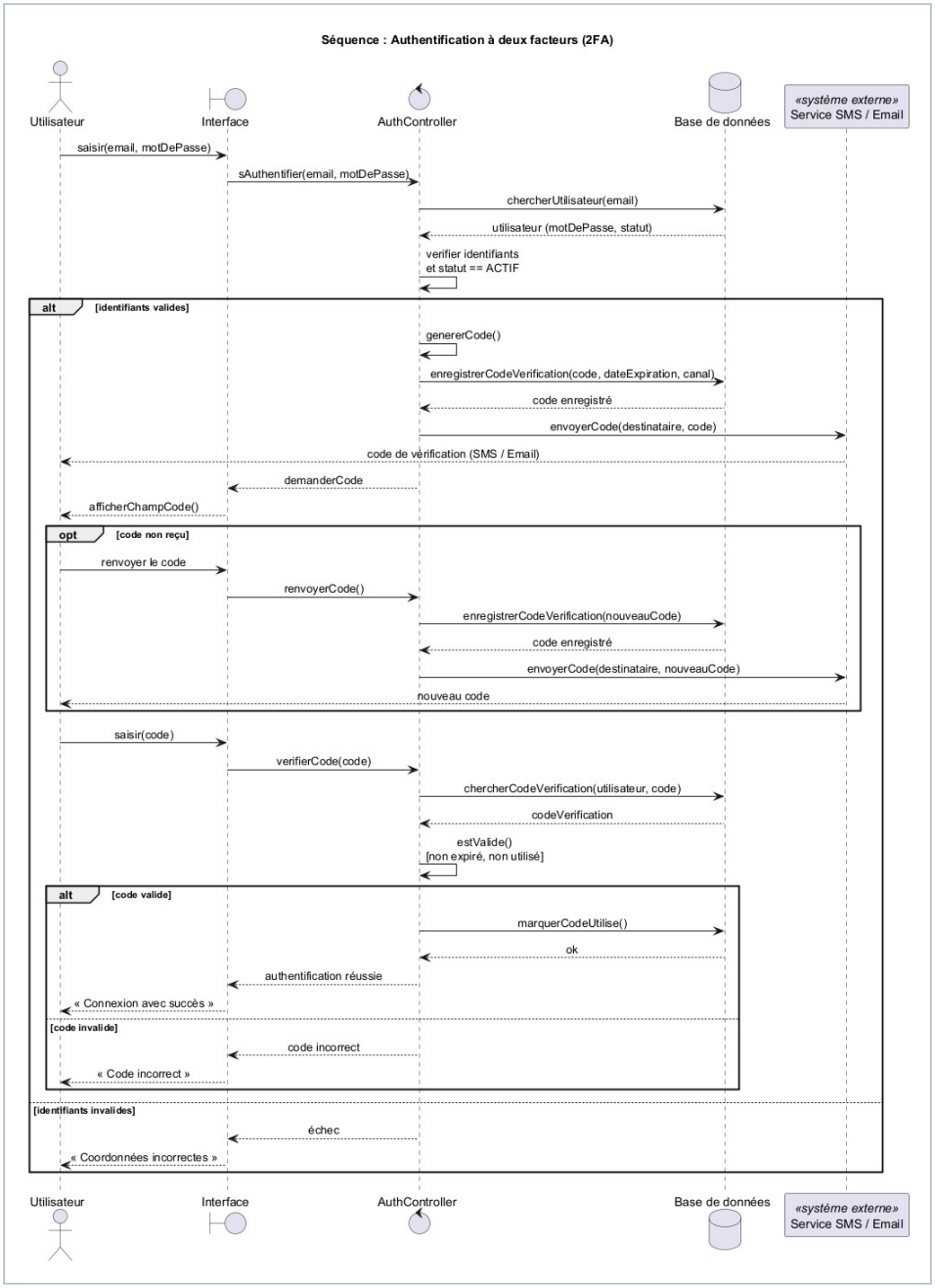


Figure 33. Diagramme de séquence : authentification à deux facteurs (2FA)

Description

Ce diagramme met en jeu l'**Utilisateur**, l'**Interface**, l'**AuthController**, la **Base de données** et le **Service SMS / Email** (système externe). Le scénario se déroule en quatre étapes :

1. L'Utilisateur saisit son email et son mot de passe. Le contrôleur recherche l'utilisateur dans la base de données et vérifie les identifiants ainsi que le statut du compte (`ACTIF`).

2. Dans le fragment **alt** principal, si les identifiants sont valides, le contrôleur génère un code, l'enregistre avec sa date d'expiration et le fait envoyer par le Service SMS / Email. L'interface affiche alors le champ de saisie du code. Sinon, le message « Coordonnées incorrectes » est affiché.
3. Dans le fragment **opt**, si le code n'est pas reçu, l'Utilisateur peut demander de renvoyer le code : un nouveau code est enregistré puis envoyé.
4. L'Utilisateur saisit le code, qui est vérifié (non expiré, non utilisé). Dans le fragment **alt** imbriqué, si le code est valide, il est marqué comme utilisé et le message « Connexion avec succès » s'affiche. Sinon, le message « Code incorrect » est affiché.

11.2 Créer un tournoi

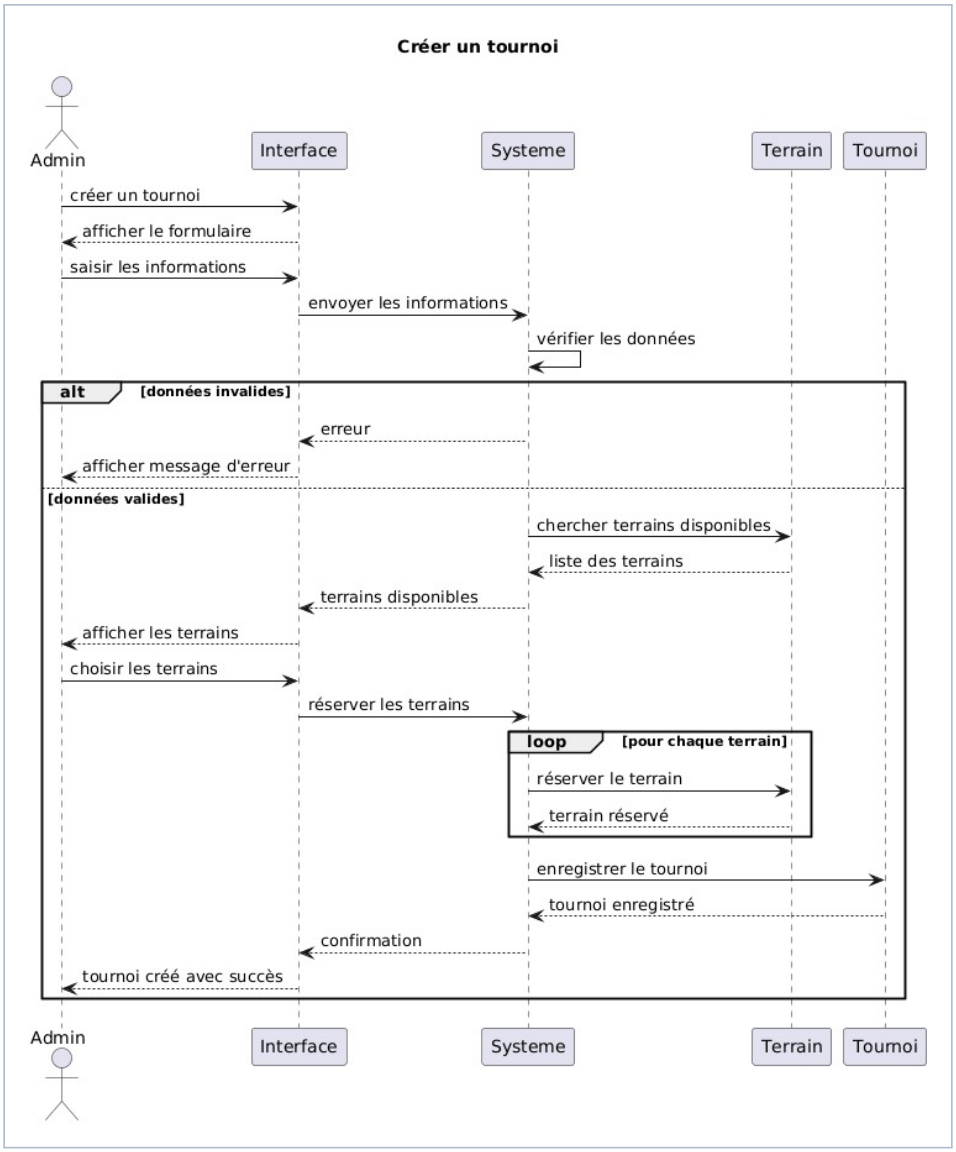


Figure 34. Diagramme de séquence : créer un tournoi

Description

Ce diagramme met en jeu l'**Admin**, l'**Interface**, le **TournoiController** et la **Base de données**. Le scénario se déroule en quatre étapes :

1. L'Admin demande la création d'un tournoi, l'interface affiche le formulaire, puis l'Admin saisit les informations (titre, dates, capacité, frais d'inscription, format). Le contrôleur vérifie les données.
2. Dans le fragment **alt**, si les données sont invalides, le contrôleur renvoie une erreur et l'interface affiche un message d'erreur à l'Admin.
3. Si les données sont valides, le contrôleur recherche les terrains disponibles dans la base de données et l'interface les affiche. L'Admin choisit alors les terrains à réserver.
4. Dans le fragment **loop**, le contrôleur réserve chaque terrain choisi (statut `RESERVE`), puis enregistre le tournoi (statut `PLANIFIEE`). Une confirmation est renvoyée à l'Admin : « Tournoi créé avec succès ».

11.3 Gérer les paiements sur place

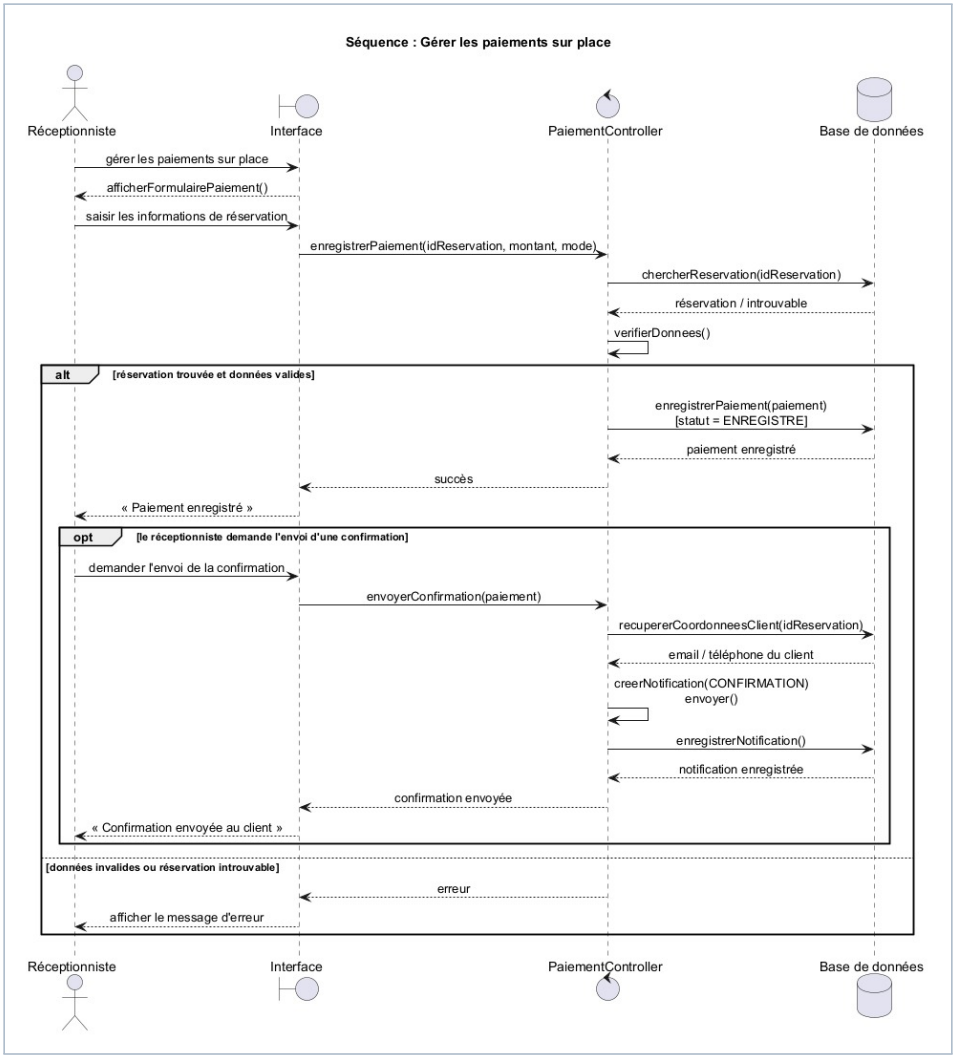


Figure 35. Diagramme de séquence : gérer les paiements sur place

Description

Ce diagramme met en jeu le **Réceptionniste**, l'**Interface**, le **PaiementController** et la **Base de données**. Le scénario se déroule en trois étapes :

1. Le Réceptionniste lance la gestion des paiements sur place, l'interface affiche le formulaire de paiement, puis le Réceptionniste saisit les informations de réservation. Le contrôleur recherche la réservation et vérifie les données.
2. Dans le fragment **alt**, si la réservation est trouvée et les données valides, le paiement est enregistré (statut `ENREGISTRE`) et le message « Paiement enregistré » s'affiche. Sinon, un message d'erreur est affiché.
3. Dans le fragment **opt**, si le Réceptionniste demande l'envoi d'une confirmation, le contrôleur récupère les coordonnées du client, crée et envoie la notification, l'enregistre

en base, puis l'interface affiche « Confirmation envoyée au client ».

12 Diagrammes d'activités

Les diagrammes d'activités décrivent l'enchaînement des actions d'un acteur pour réaliser une fonctionnalité, avec les décisions et les traitements parallèles.

12.1 Coach : consulter son planning

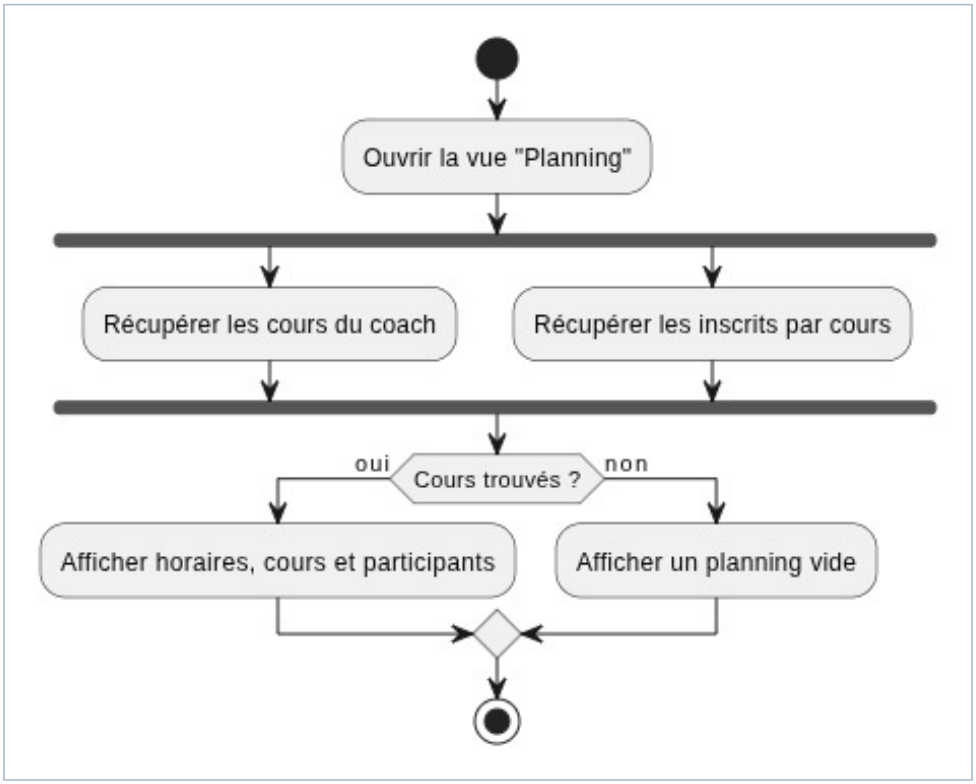


Figure 36. Diagramme d'activités : consulter son planning (Coach)

Le **Coach** ouvre la vue « Planning ». Le système récupère **en parallèle** les cours du coach et les inscrits par cours. Si des cours sont trouvés, il affiche les horaires, les cours et les participants. Sinon, il affiche un planning vide.

12.2 Coach : gérer ses cours

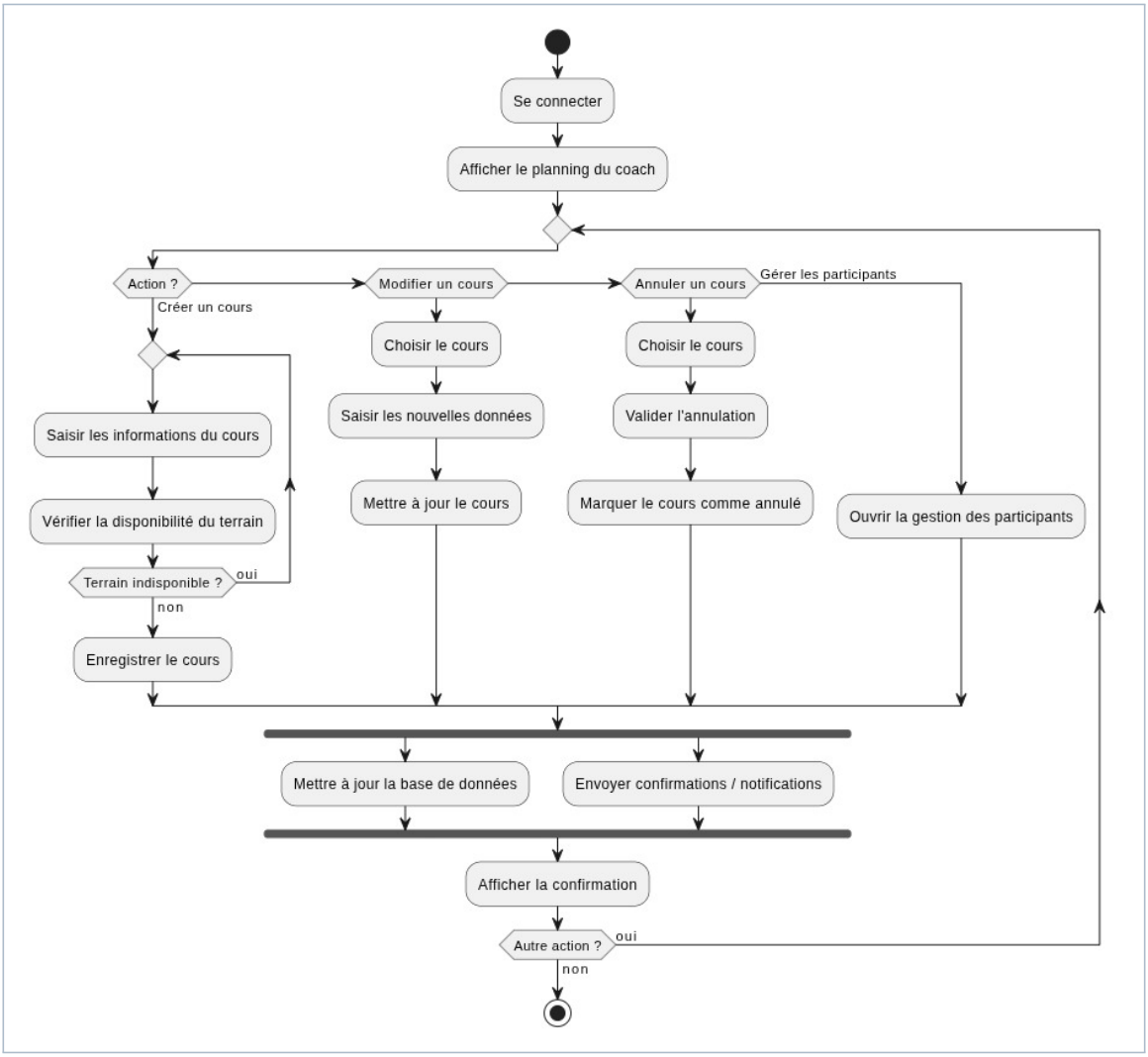


Figure 37. Diagramme d'activités : gérer ses cours (Coach)

Après s'être connecté, le **Coach** voit son planning, puis choisit une action parmi quatre :

- **Créer un cours** : saisir les informations du cours et vérifier la disponibilité du terrain. Si le terrain est indisponible, la saisie est reprise. Sinon, le cours est enregistré.
- **Modifier un cours** : choisir le cours, saisir les nouvelles données et mettre à jour le cours.
- **Annuler un cours** : choisir le cours, valider l'annulation et marquer le cours comme annulé.
- **Gérer les participants** : ouvrir la gestion des participants.

Ensuite, le système met à jour la base de données et envoie les confirmations et notifications **en parallèle**, puis affiche la confirmation. Le Coach peut alors effectuer une autre action ou terminer.

12.3 Administrateur

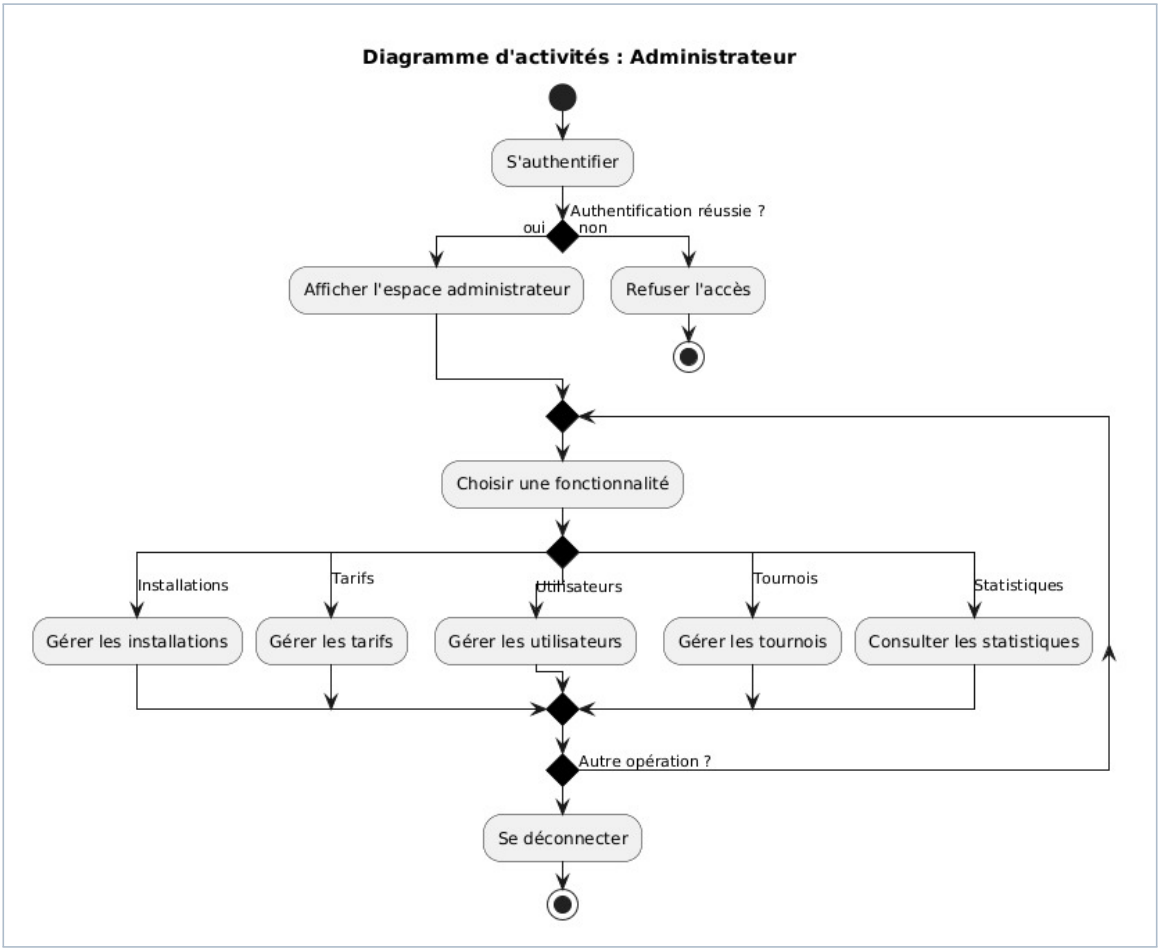


Figure 38. Diagramme d'activités : Administrateur

L'**Administrateur** s'authentifie. Si l'authentification échoue, l'accès est refusé. Si elle réussit, l'espace administrateur est affiché et l'Administrateur choisit une fonctionnalité : gérer les installations, les tarifs, les utilisateurs ou les tournois, ou consulter les statistiques. Il peut ensuite effectuer une autre opération, ou se déconnecter.

12.4 Réceptionniste : gérer les paiements sur place

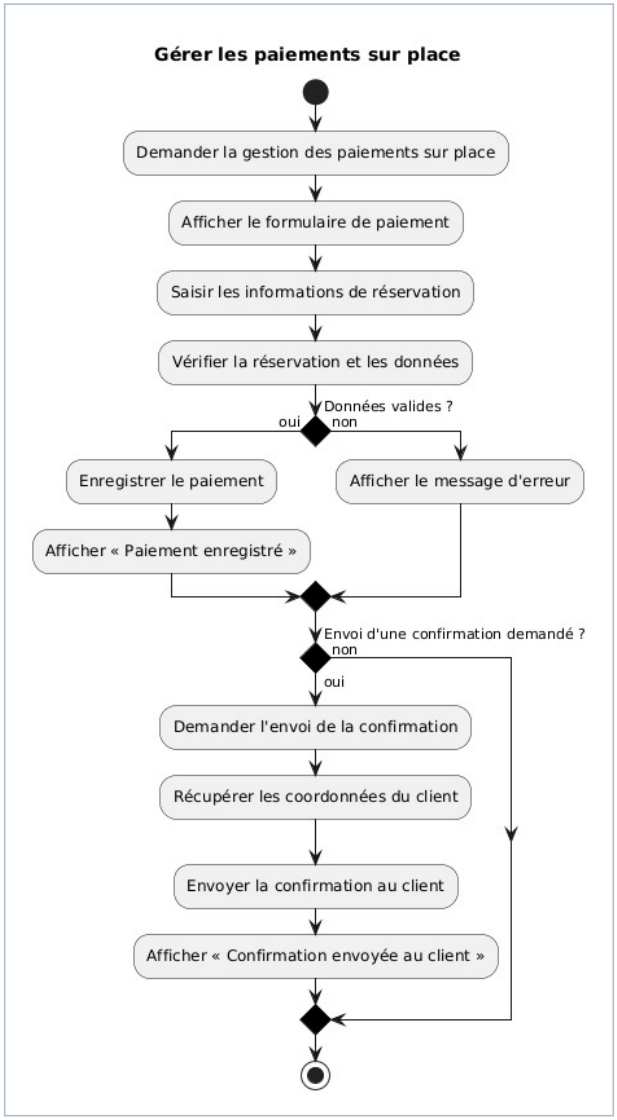


Figure 39. Diagramme d'activités : gérer les paiements sur place (Réceptionniste)

Le **Réceptionniste** demande la gestion des paiements sur place, puis le système affiche le formulaire de paiement. Après la saisie des informations de réservation, le système vérifie la réservation et les données. Si elles sont valides, le paiement est enregistré et le message « Paiement enregistré » s'affiche. Sinon, un message d'erreur est affiché. Si l'envoi d'une confirmation est demandé, le système récupère les coordonnées du client, lui envoie la confirmation et affiche « Confirmation envoyée au client ».